

Vorlesung (WS 2016/17) *Softwarekonstruktion*

Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

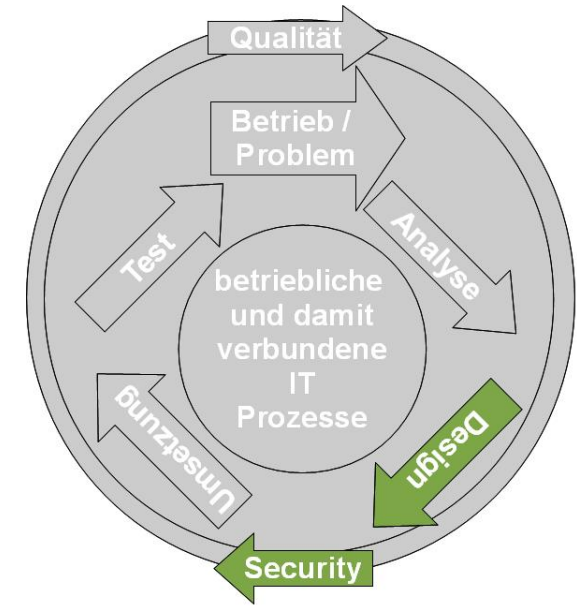
Teil 1.3: Petrinetze und nebenläufige Programmierung

v. 7.11.2016

Einordnung

1.4 Petrinetze

- Modellgetriebene SW-Entwicklung
 - Einführung
 - Modellbasierte Softwareentwicklung
 - OCL
 - Petrinetze
 - Eclipse Modeling Framework (EMF)
- Qualitätsmanagement
- Testen



Inkl Beiträge von Prof. Volker Gruhn, Jutta Mülle und Dr. Silvia von Stackelberg.era

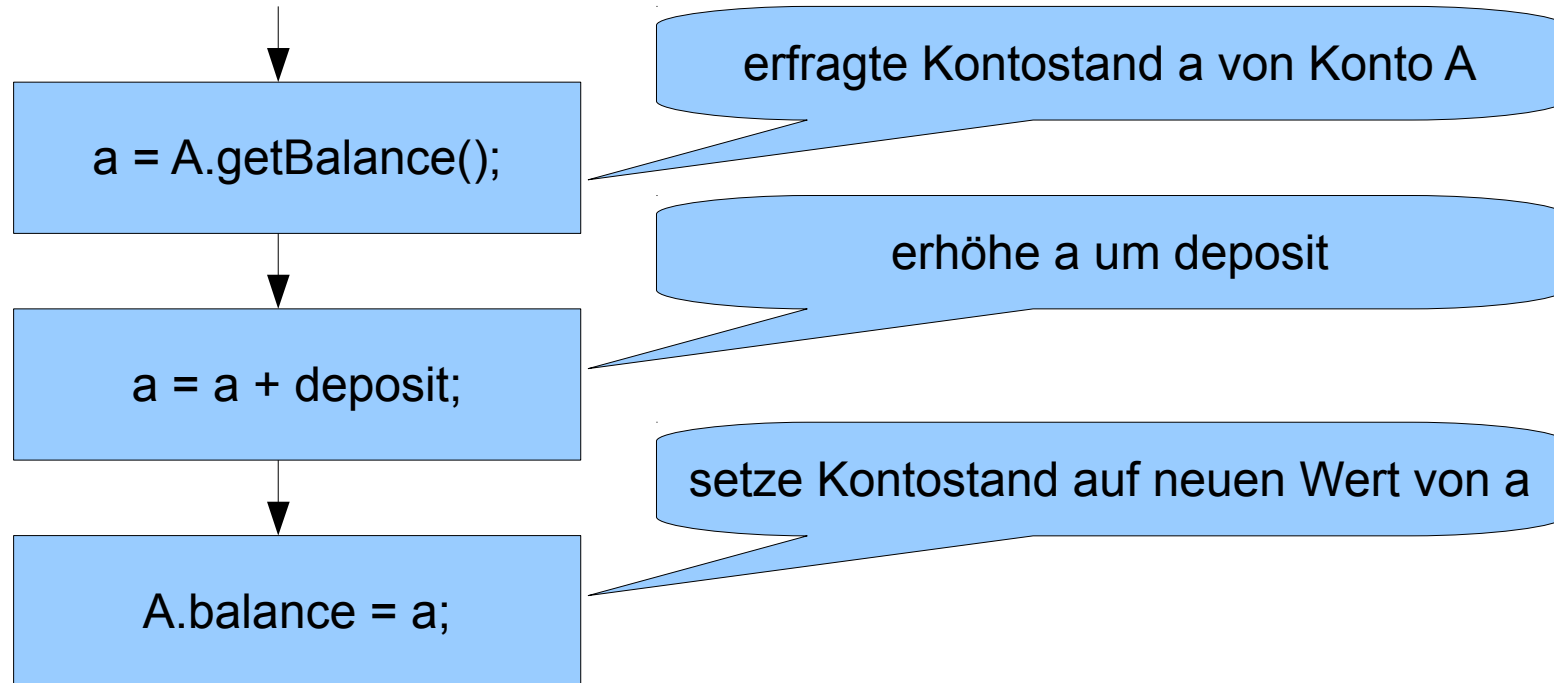
Literatur (s. Vorlesungswebseite):

[Rei10] W. Reisig: Petrinetze. Vieweg, 2010.

- Teil I

- Grundlegendes Verständnis der Notwendigkeit für die Spezifikation von nebenläufigen Systemen
- Kenntnisse im Umgang mit Petri-Netzen als Spezifikationsformalismus für nebenläufige Systeme
- Übersicht über Abstraktions- und Programmiermodelle für nebenläufige Programme

Der Stand eines Konto wird abgefragt, eine Summe „deposit“ eingezahlt, die Kontostandartvariable um einen Betrag „deposit“ erhöht und der neue Wert als neuer Kontostand eingetragen:



Nebenläufig = mehrere Prozesse werden zu überlappenden Zeitspannen ausgeführt

Parallel = mehrere Prozesse werden zur gleichen Zeitspanne ausgeführt

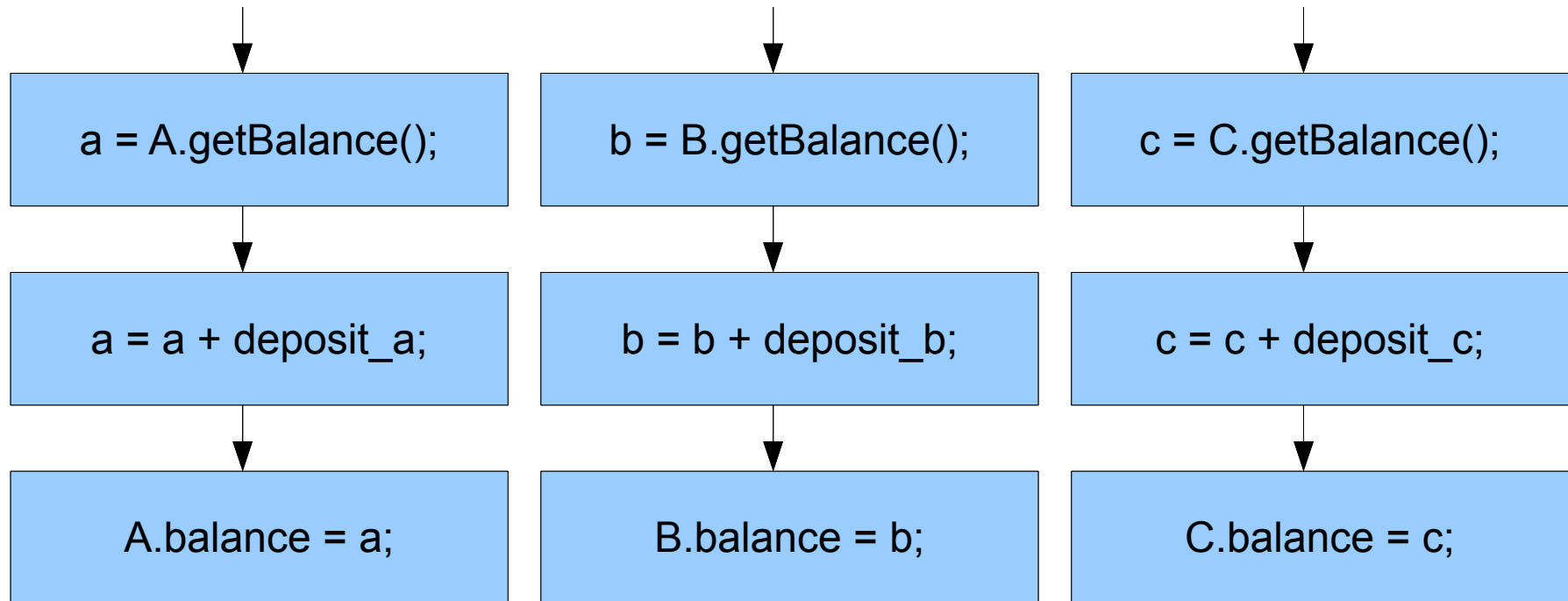
→ Parallelismus auf Single-core unmöglich

Herlihy, Maurice: The Art of Multiprocessor Programming Morgan Kaufmann, 2012

Betriebssystemunterstützung für nebenläufige Programme:

- Multitasking (kooperativ oder präemptiv)
- Time sharing (nebenläufige Variante der sequentiellen Batch-Jobs)
- Prozesse
- Threads

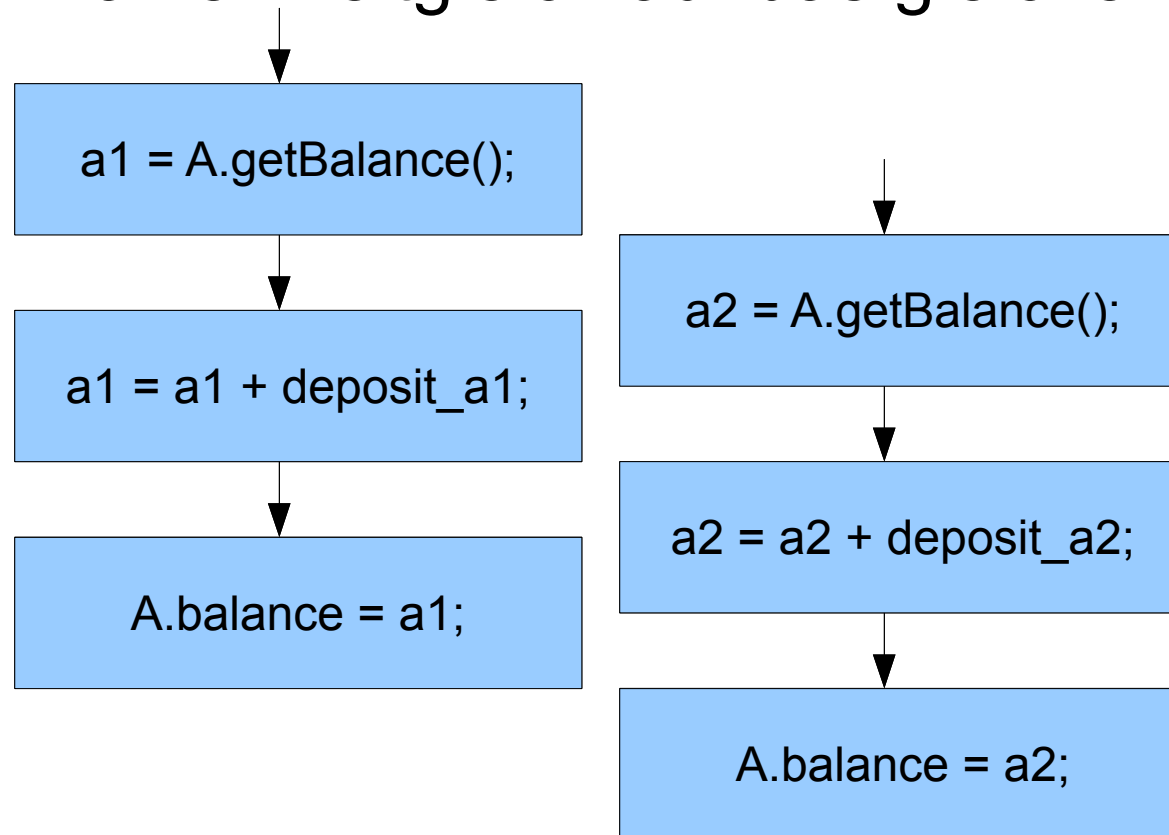
Einzahlungen können auch parallel verlaufen:



Innerhalb eines Computers wird dies „multi-threading“ genannt. Ein Thread kann eine Aufgabe unabhängig von anderen Threads erfüllen.

- Lokale Variable, formale Parameter und Parameter von Ausnahmebehandlern:
 - sind lokal für jeden Thread;
 - haben keinen Einfluss auf andere Threads.
- Instanzvariable, statische Variable, Elemente von Reihungen:
 - residieren in einem globalen Speicher.
 - Änderungen können die Werte anderer Threads beeinflussen.

Zwei Kunden zahlen zeitgleich auf das gleiche Konto ein:



Erster Eintrag verloren → „Race Condition“, Interferenz zwischen Threads

- Eine **Race Condition** ist eine Situation, in der zwei oder mehr Threads gemeinsame Daten schreiben oder lesen und das Endergebnis vom Scheduling der Threads abhängt.
- Race Conditions sind gefürchtet, da sie zu **schwer vorhersagbarem und schwer reproduzierbarem Programmverhalten** führen.
- Eine Race Condition manifestiert sich oft nur sehr selten (etwa 1 Mal pro 1000 Programmabläufe) und ist daher **durch Testen kaum aufzuspüren**.

- Interaktion zwischen Threads kann zu unerwünschtem Verhalten führen.
- Zwei Arten von Eigenschaften nebenläufiger Programme:
- **Sicherheit** = Eigenschaft, dass nie etwas Unerwünschtes geschieht
- **Lebendigkeit** = Eigenschaft, dass immer irgendetwas geschieht
- **Sicherheitsfehler** führen zu unerwünschtem Laufzeitverhalten
- **Lebendigkeitsprobleme** führen dazu, dass das Programm anhält, obwohl noch nicht alle Threads terminiert haben.

Sicherheitsprobleme basieren häufig auf

- **Lese / Schreibkonflikten**

- Der Zugriff eines lesenden Kunden auf ein Objekt während eines noch nicht beendeten Schreibvorgangs kann zum lesen inkorrektur Daten führen
- Beispiel: Lesen einer Reihung während alle Feldelemente verdoppelt werden

- **Schreib/Schreibkonflikte**

- Inkonsistente Zuweisung an Variablen durch nebenläufiges Ausführen von zustandsändernden Methoden

Probleme:

- **Race condition** = Ergebnis der Operation hängt vom zeitlichen Verhalten bestimmter Einzeloperationen ab. → Schwer auffindbare Programmfehler.
- **Deadlocks** = zyklische Wartesituation auf Ressourcenzuteilung
- **Resource starvation** = Prozess erhält wiederholt (potentiell unendlich oft) keine Zuteilung auf benötigte Ressourcen

Daher Zugriffsregelung auf Ressourcen und Scheduling notwendig.

Programmiersprachenunterstützung für Nebenläufigkeit

- Kanäle (Channels) = Kanal mit zwei Endpunkten zur Kommunikation (typsicher (Erlang) oder typunsicher (Go))
- Coroutinen = Multithreading-sichere Routinen
- Futures und Promises = Objekte als Platzhalter für zukünftige Resultate
 - `future factorial(1000000000000)`

Shared Memory

- Nebenläufige Prozesse teilen sich gemeinsamen Speicherbereich zur Kommunikation
 - Sperren notwendig durch Mutex, Semaphore, Monitor, etc.

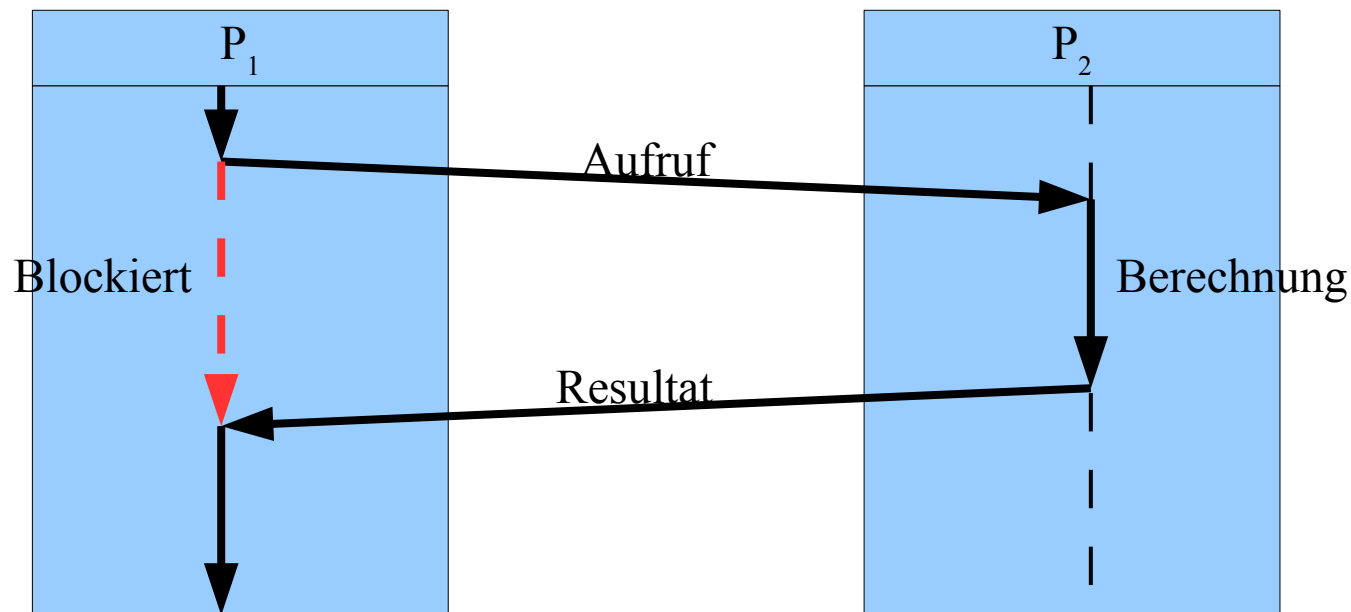
Message passing

- Nebenläufige Prozesse tauschen Nachrichten aus
 - synchron/asynchron
 - verlässlich/unverlässlich

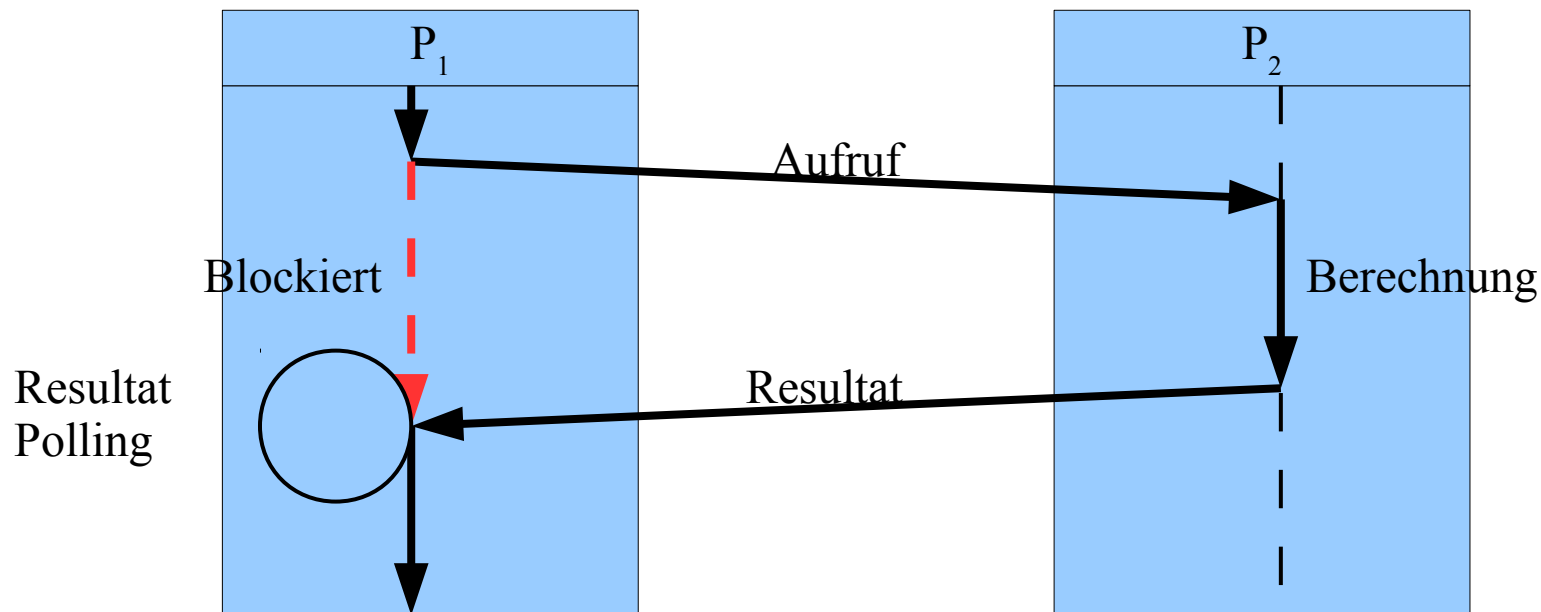
Nebenläufige Einheiten (Actors) kommunizieren nur über Nachrichtenaustausch (verbreitet in funktionalen Sprachen)

- Actor hat Posteingang, Adresse und Verhalten
- Nachrichten heißen Ereignisse
- Erlaubte Reaktionen auf Ereignisse
 - Nachrichten an sich selbst oder andere Actors versenden
 - Neue Actors erzeugen
 - Eigenes Verhalten ändern
- Nachteil: Nachrichten und interner Speicher immer Kopie, da kein shared memory

- Prozess P_1 ruft Service von P_2 synchron auf
- P_2 benötigt Zeit um Resultat bereitzustellen
- Was macht P_1 in der Zwischenzeit?



- Prozess P_1 ruft Service von P_2 asynchron auf
- P_2 benötigt Zeit um Resultat bereitzustellen
- P_1 arbeitet weiter und muss P_2 Resultat pollen



- **Proxy** für **Ergebnis**, das noch nicht bekannt ist
- Future = Rückgabewert einer asynchronen Funktion (promise)
- Zugriff möglich, wenn Berechnung abgeschlossen
- Futures werden als Argumente an andere Prozeduraufrufe **weitergereicht**
- Auswertung beginnt, wenn Ergebnis des Futures vorhanden
- Konzept: Future hat get- oder join-Funktion, die so lange **blockiert**, bis Ergebnis vorliegt, und dieses dann zurückliefert

```
interface ArchiveSearcher { String search(String
target); }
class App {
    ExecutorService executor = ...
    ArchiveSearcher searcher = ...
    void showSearch(final String target)
        throws InterruptedException {
        Future<String> future
        = executor.submit(new Callable<String>() {
            public String call() {
                return searcher.search(target);
            }
        });
        displayOtherThings(); //do other things
        try {
            displayText(future.get()); // use future
        } catch (InterruptedException ex) { cleanup(); return; }
    }
}
```

Quelle: Oracle Java™ Platform Standard Ed. 8 Documentation

Vorheriger Abschnitt:

Präzise Definition des Ausführungsverhaltens notwendig für Ausführung,
aber OCL liefert nur Constraints.

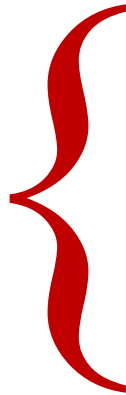
Verschiedene Ansätze: Abstract State Machines, Petrinetze, ...

- Z.B.: **Ausführungssemantik** von **UML 2-Aktivitätsdiagrammen** mit Petrinetzen definiert.

Dieser Abschnitt:

Petrinetze → Verwendet in zentralem Ansatz für **Process-Mining**.

1.4 Petrinetze



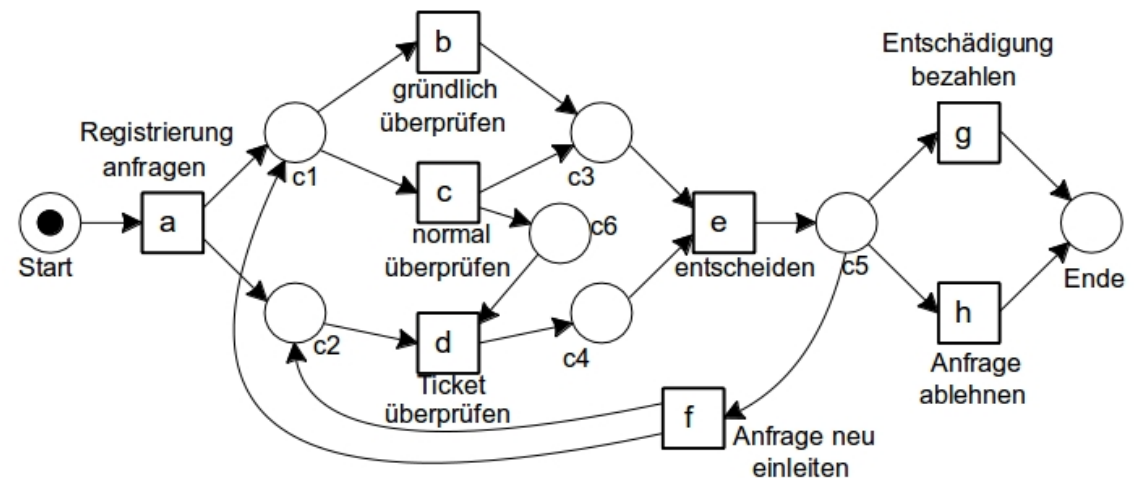
Petrinetz Syntax

Ausführung

Analyse von Systemen

Workflow-Netze

- Modellierung, Analyse, Simulation von dynamischen Systemen mit **nebenläufigen** und **nichtdeterministischen** Merkmalen.
- Erlauben die Beschreibung von **Kontroll- und Datenfluss**.
- Benannt nach Carl Adam Petri (Dissertation "Kommunikation mit Automaten", 1962).



Vorsicht: Existieren heute viele Varianten.

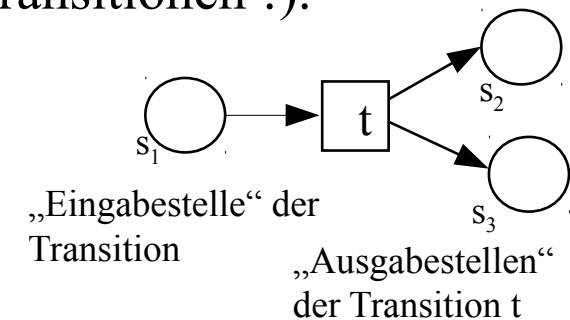
Statische Komponente: Bipartiter gerichteter Graph, bestehend aus:

- zwei Sorten von **Knoten**:
 - **Stelle** („S-Elemente“): Zwischenablage von Informationen
 - **Transitionen** („T-Elemente“): Verarbeitung von Informationen



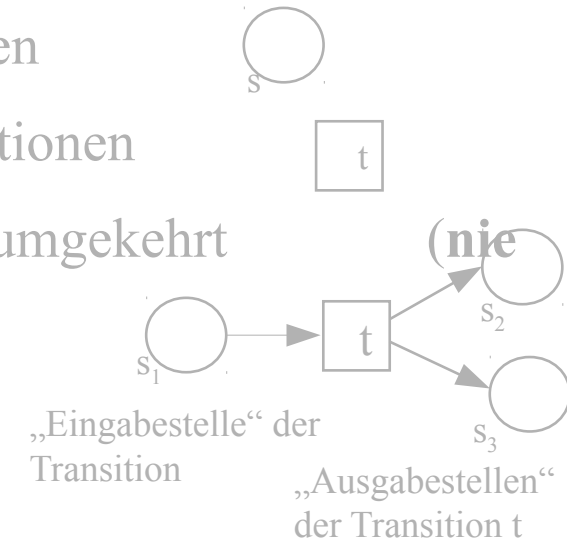
Statische Komponente: Bipartiter gerichteter Graph, bestehend aus:

- zwei Sorten von **Knoten**:
 - **Stelle** („S-Elemente“): Zwischenablage von Informationen
 - **Transitionen** („T-Elemente“): Verarbeitung von Informationen
- **Kanten** („Bogen“): verbinden Stellen mit Transitionen oder umgekehrt (**nie** Stellen mit Stellen oder Transitionen mit Transitionen !).



Statische Komponente: Bipartiter gerichteter Graph, bestehend aus:

- zwei Sorten von **Knoten**:
 - **Stelle** („S-Elemente“): Zwischenablage von Informationen
 - **Transitionen** („T-Elemente“): Verarbeitung von Informationen
- **Kanten** („Bogen“): verbinden Stellen mit Transitionen oder umgekehrt Stellen mit Stellen oder Transitionen mit Transitionen !).



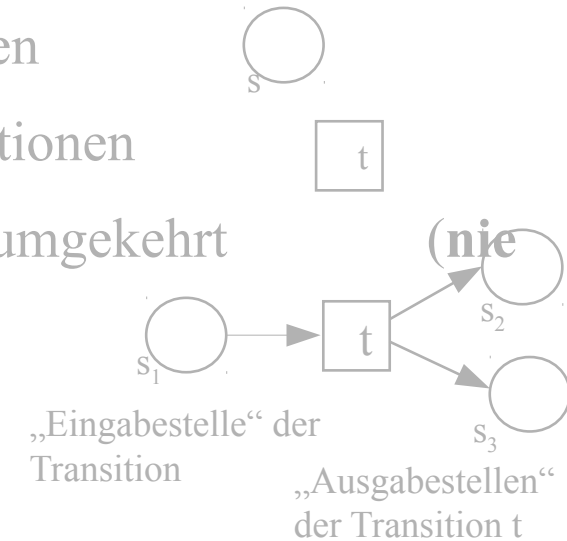
Dynamische Komponente:

- **Marken** („Token“): Stellen werden mit Objekten belegt.
 - Durchlauf der Token durch Petrinetz beschreibt **Verhalten** des Systems.



Statische Komponente: Bipartiter gerichteter Graph, bestehend aus:

- zwei Sorten von **Knoten**:
 - **Stelle** („S-Elemente“): Zwischenablage von Informationen
 - **Transitionen** („T-Elemente“): Verarbeitung von Informationen
- **Kanten** („Bogen“): verbinden Stellen mit Transitionen oder umgekehrt Stellen mit Stellen oder Transitionen mit Transitionen !).

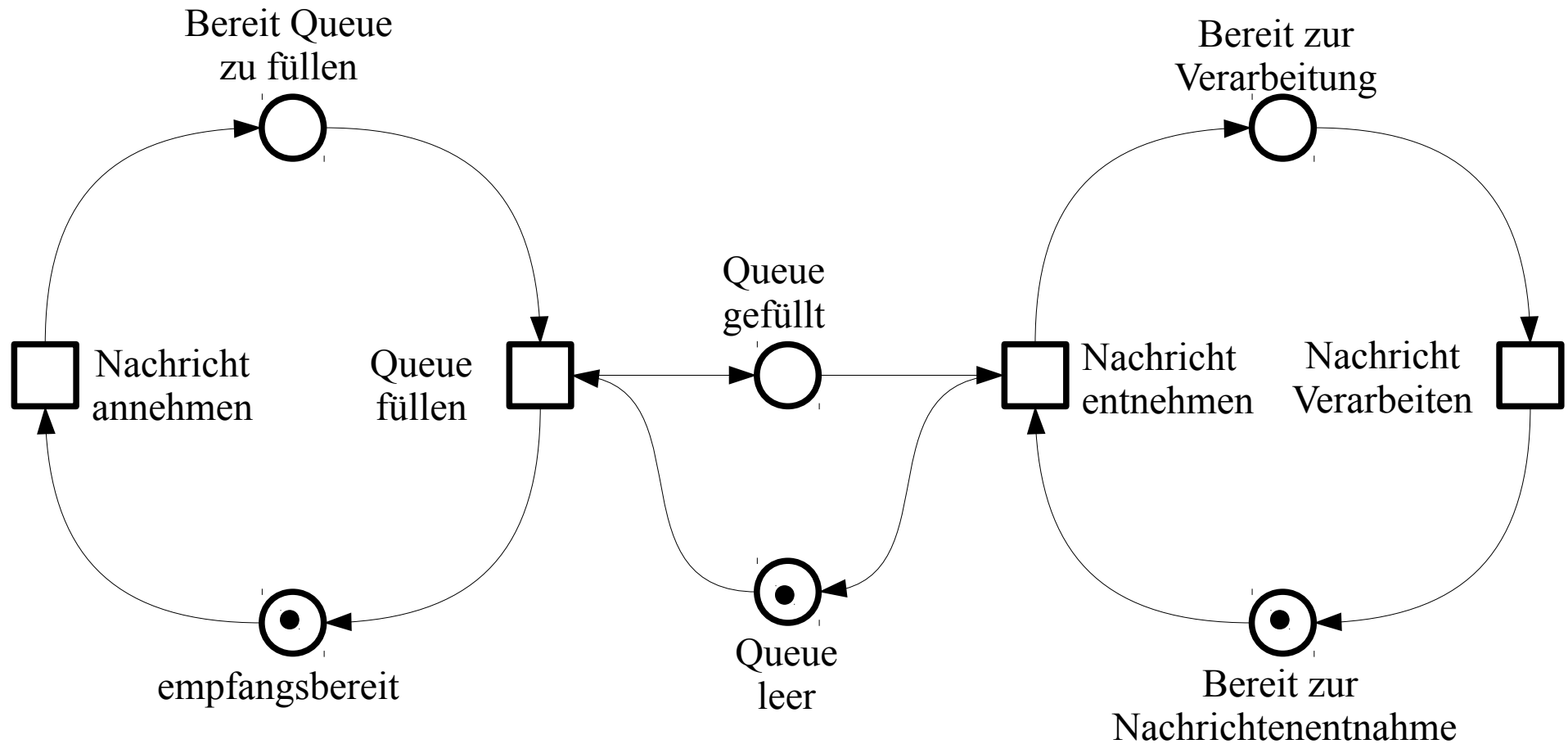


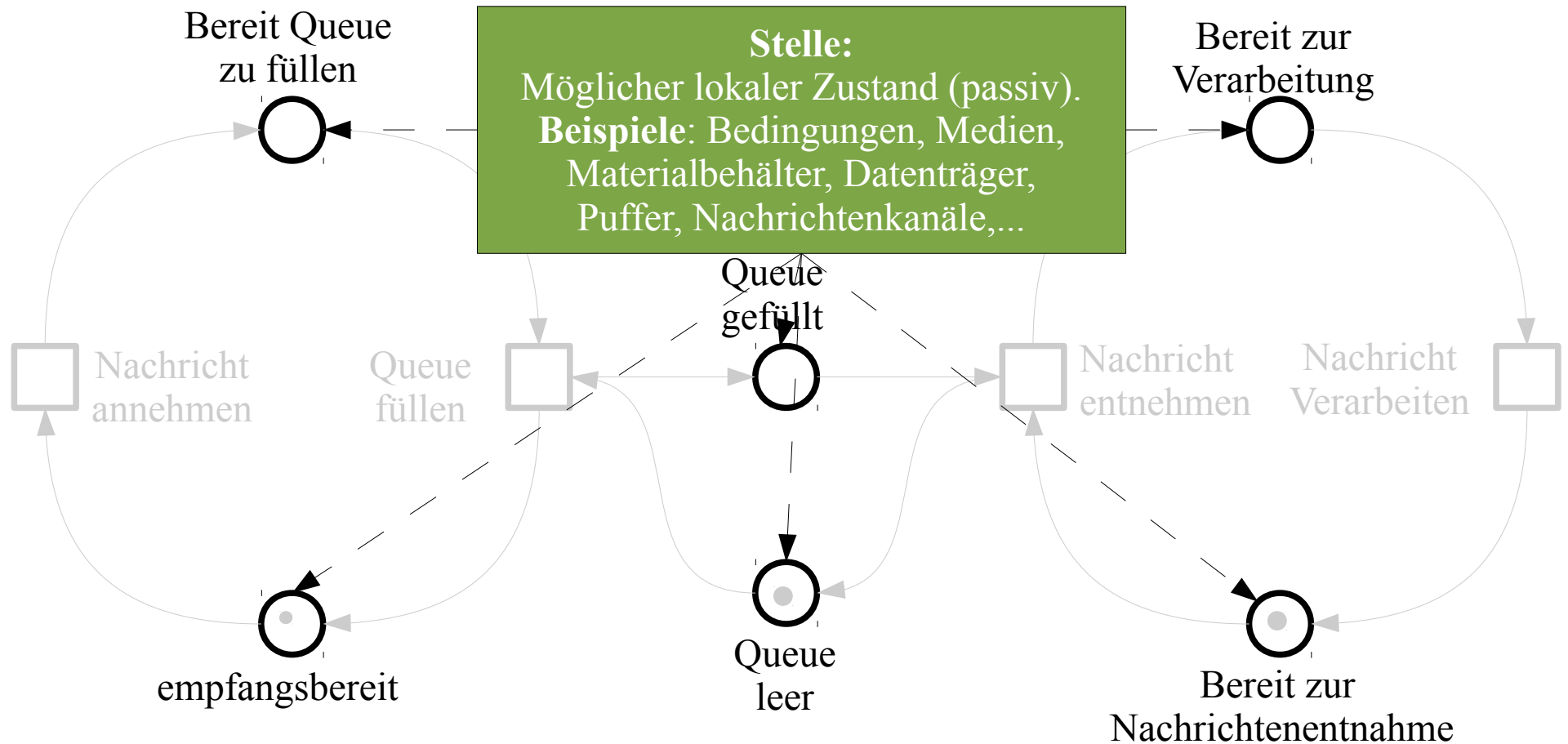
Dynamische Komponente:

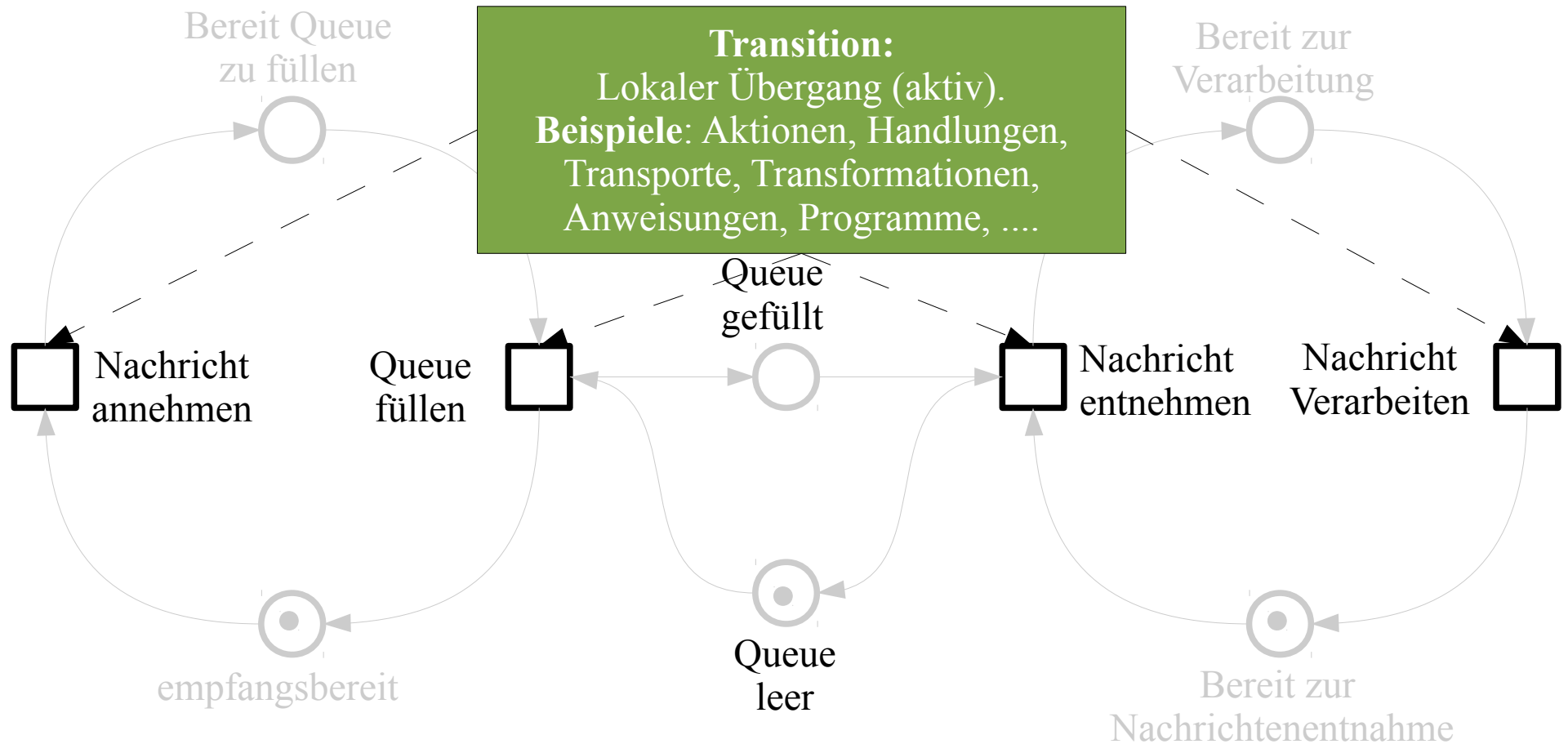
- **Marken** („Token“): Stellen werden mit Objekten belegt.
 - Durchlauf der Token durch Petrinetz beschreibt **Verhalten** des Systems.
 - Hier: nur **ungetypte, nicht unterscheidbare** Marken „**Stellen/Transitions-Netz**“).

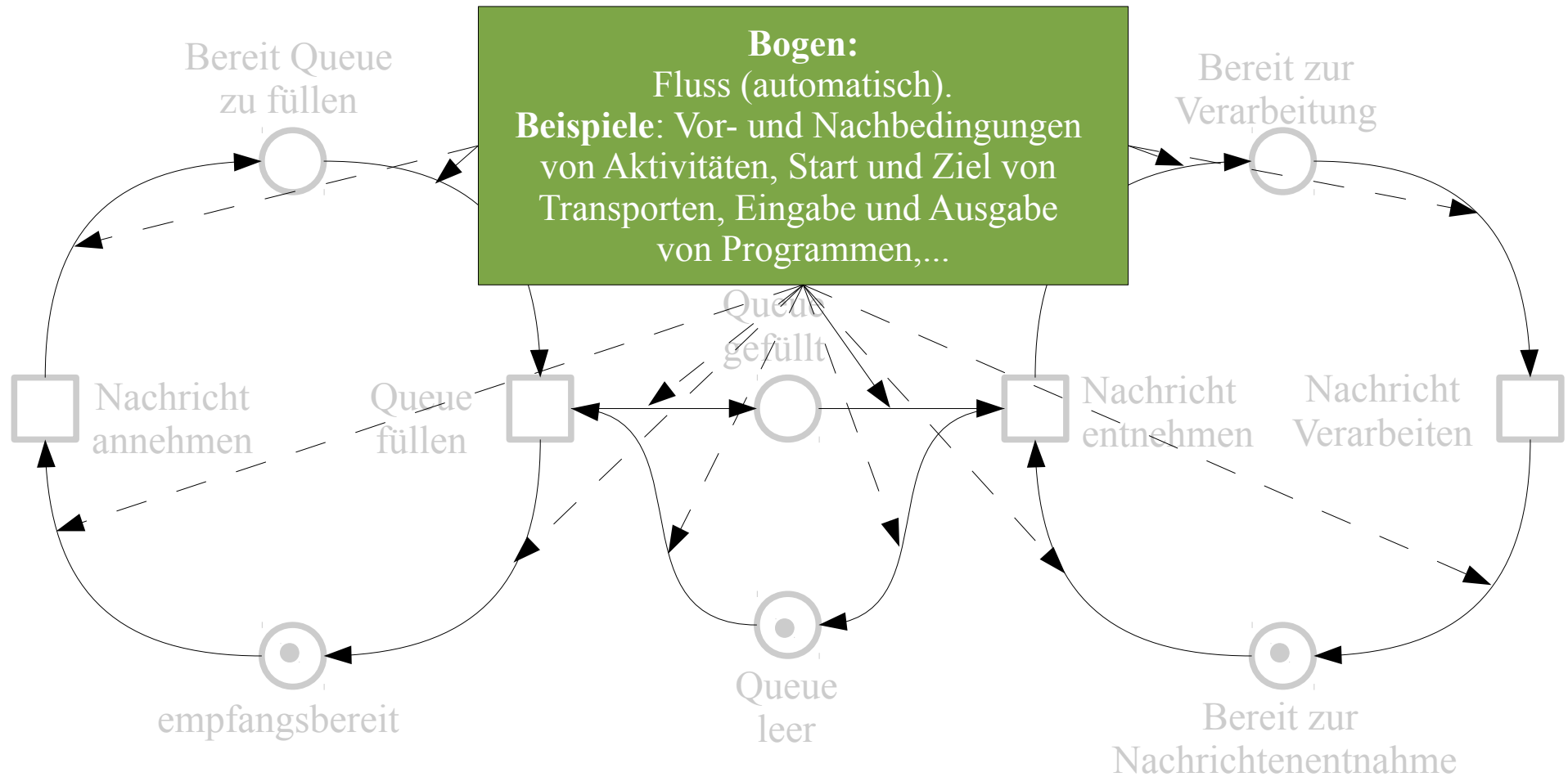


(→

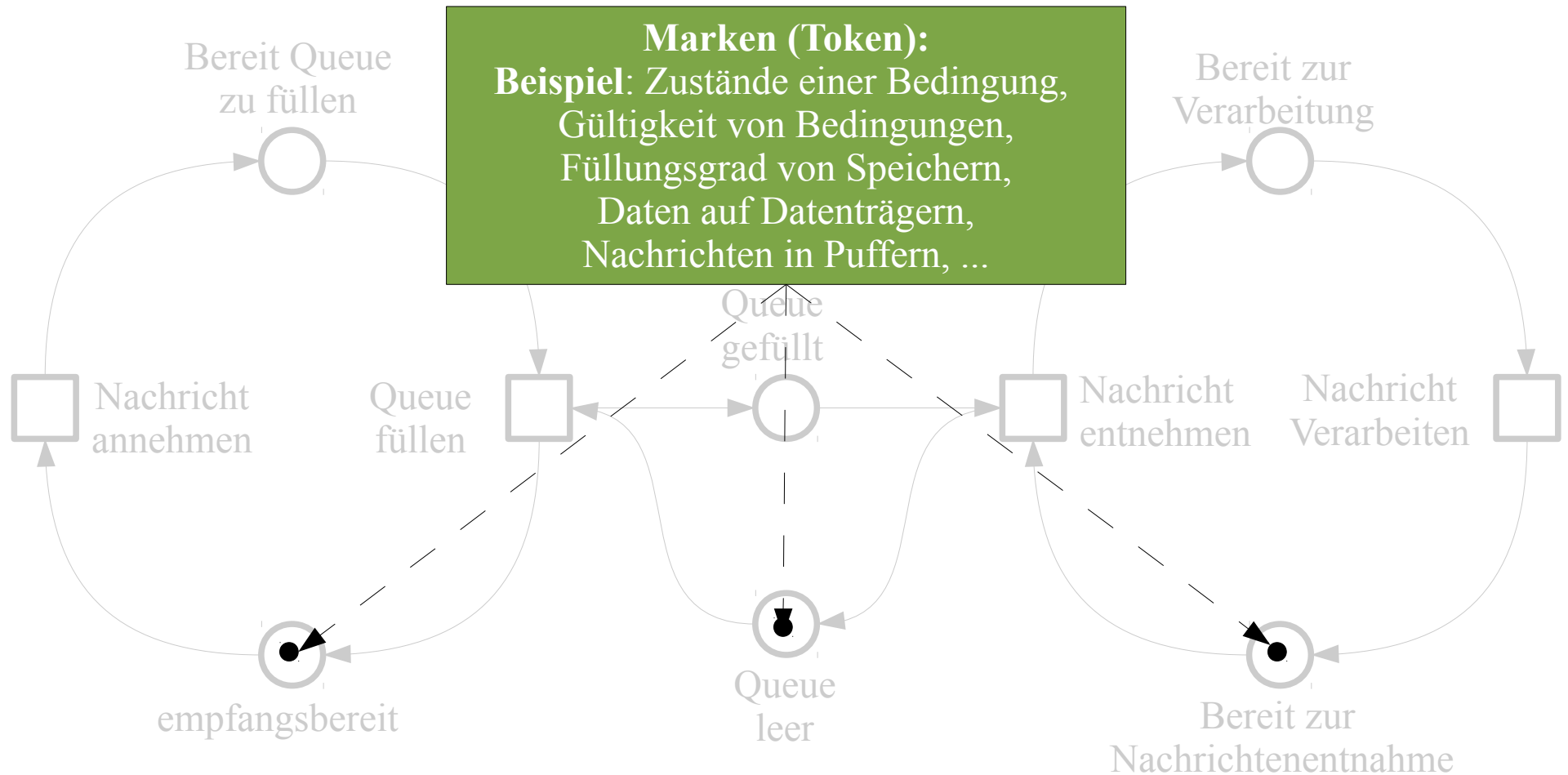




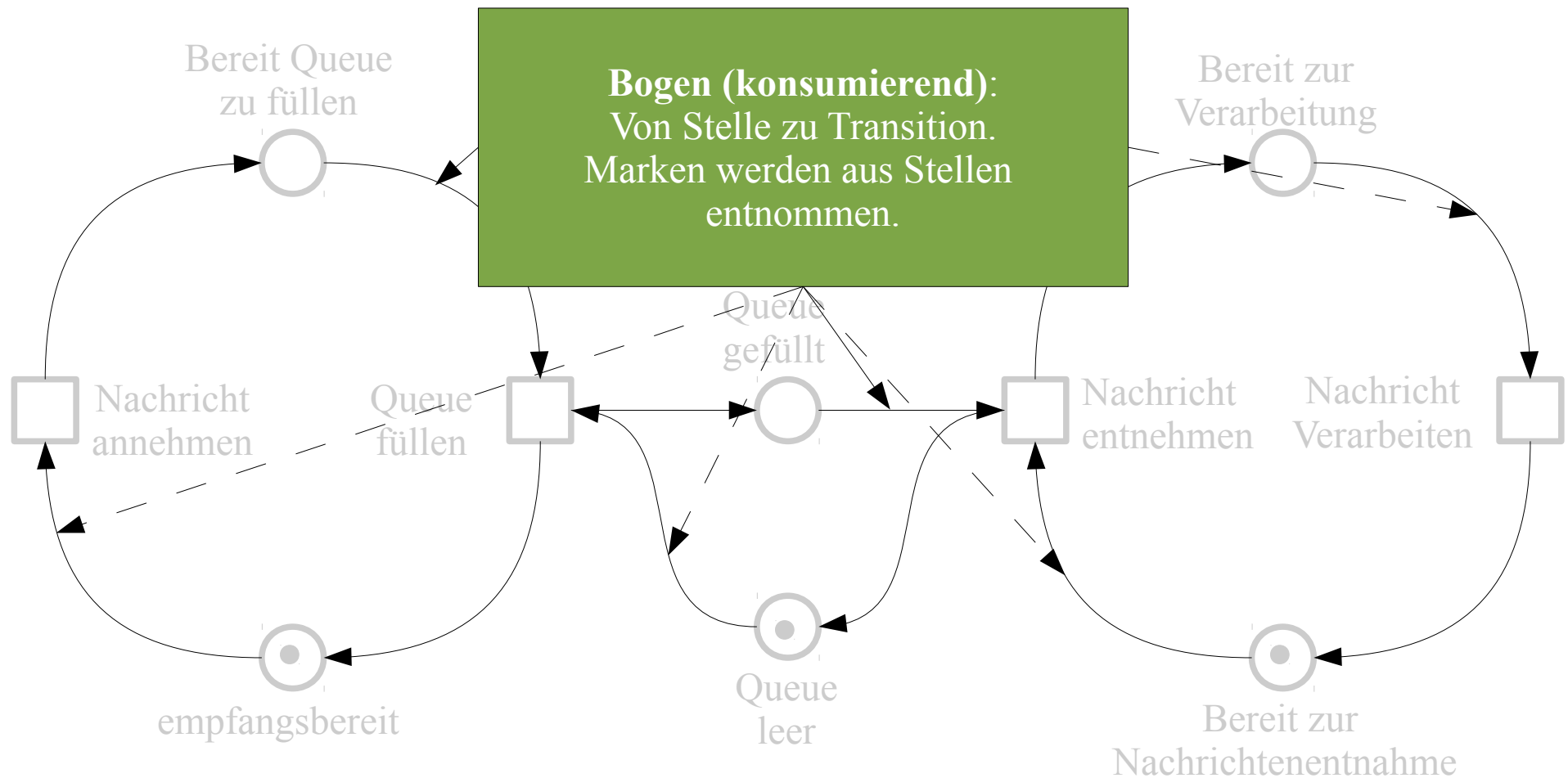




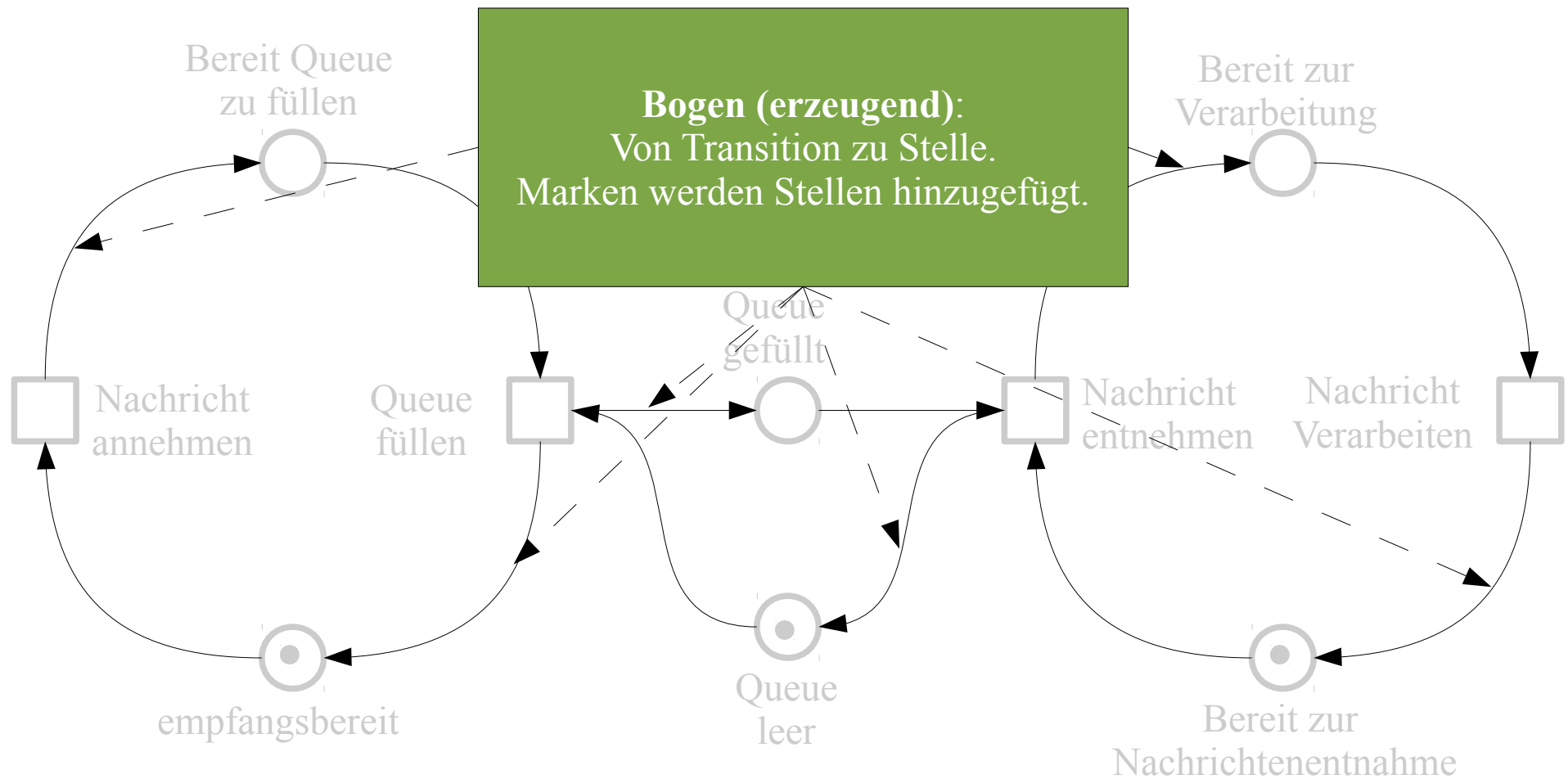
Petrinetz-(Ausführungs-)Syntax: Marken (Token)

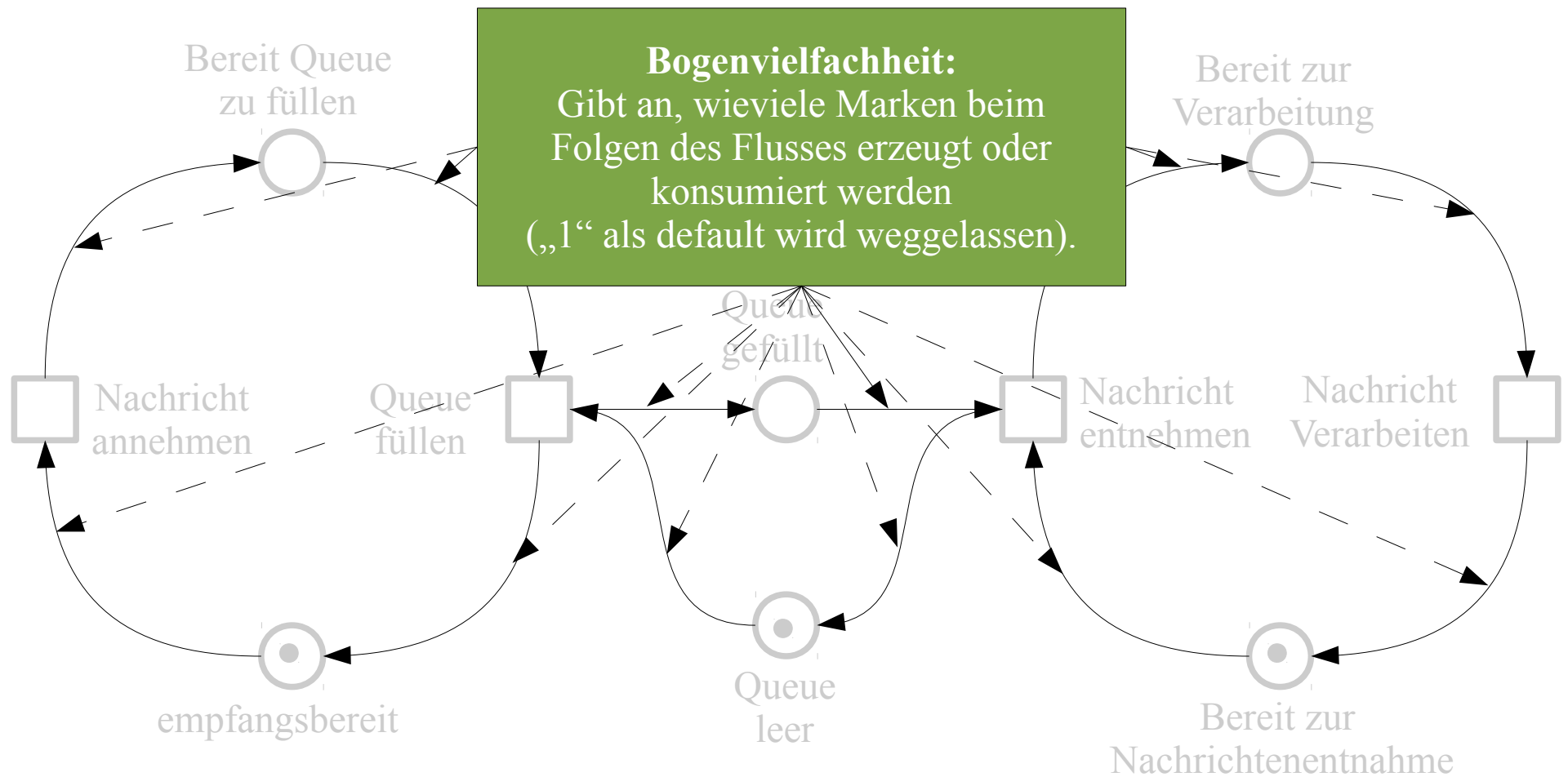


Petrinetz-Syntax: Bogen (konsumierend)



Petrinetz-Syntax: Bogen (erzeugend)





Gegeben:

- S: endliche Menge von **Stellen**
- T: endliche Menge von **Transitionen**

mit: $S \neq \emptyset$,

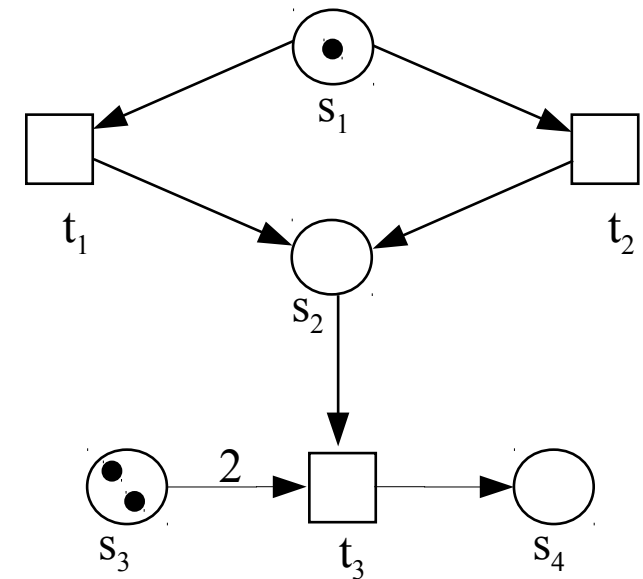
$T \neq \emptyset$ und

$S \cap T = \emptyset$

- F: Menge von **Bögen** („Kanten“, „Flüsse“)

mit: $F \subseteq (S \times T) \cup (T \times S)$

als binäre Relation („Flussrelation“)



$S = \{s_1, s_2, s_3, s_4\}$

$T = \{t_1, t_2, t_3\}$

$F = \{(s_1, t_1), (s_1, t_2), (t_1, s_2),$
 $(t_2, s_2), (s_2, t_3), (s_3, t_3), (t_3, s_4)\}$

Gegeben:

- **K: Kapazität** („Fassungsvermögen der Stellen“)

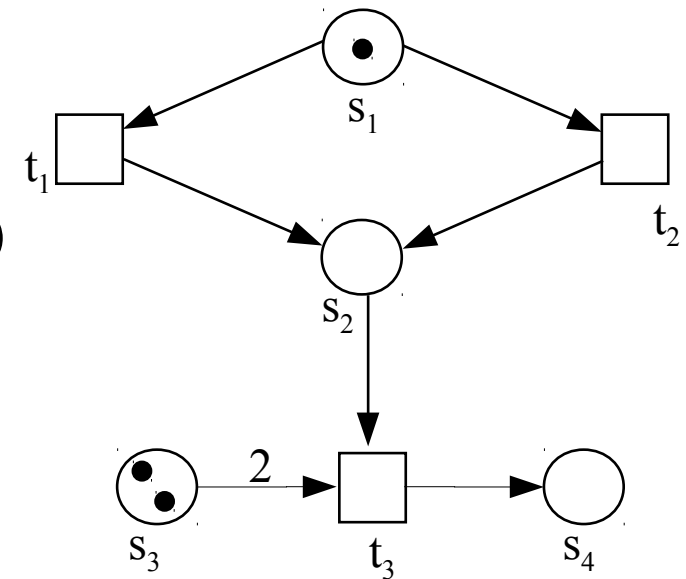
mit: $K : S \rightarrow \mathbb{N} \cup \{\infty\}$

Default-Kapazität

- **W: Bogenvielfachheit** („Gewicht der Kanten“)

mit: $W : F \rightarrow \mathbb{N} \setminus 0$

Default-Kantengewicht 1



$$K(s_1) = K(s_2) = K(s_3) = K(s_4) = \infty$$

$$\begin{aligned} W(s_1, t_1) &= W(s_1, t_2) = W(s_2, t_1) = \\ W(s_2, t_2) &= W(s_2, t_3) = W(t_3, s_4) = 1, \\ W(s_3, t_3) &= 2 \end{aligned}$$

sowie

- M_0 : **Globaler Startzustand**
(„Anfangsmarkierung“)

mit: $M_0: S \rightarrow \mathbb{N}$

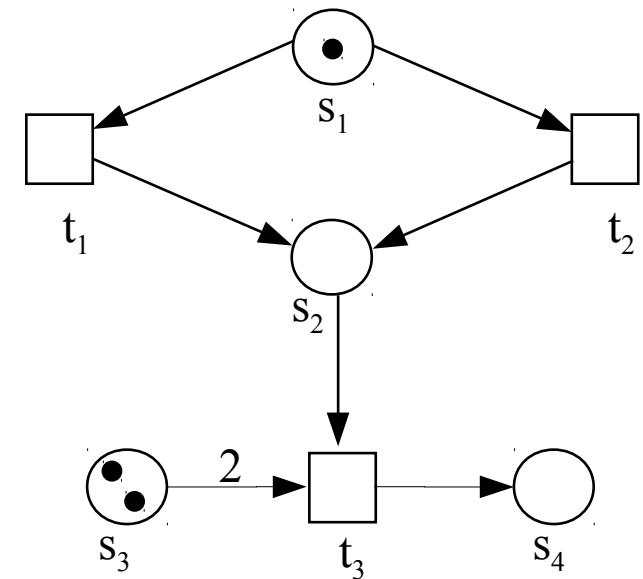
(S, T, F, W, K, M_0) ist **Petrinetz** (sog. S/T-Netz)

- **Markierungen M** :

mit: $M: S \rightarrow \mathbb{N}$

Markierungen müssen Kapazitäten respektieren,

d.h. für jede Stelle $s \in S$ gilt: $M(s) \leq K(s)$



$$M_0(s_1) = 1,$$

$$M_0(s_2) = 0,$$

$$M_0(s_3) = 2,$$

$$M_0(s_4) = 0$$

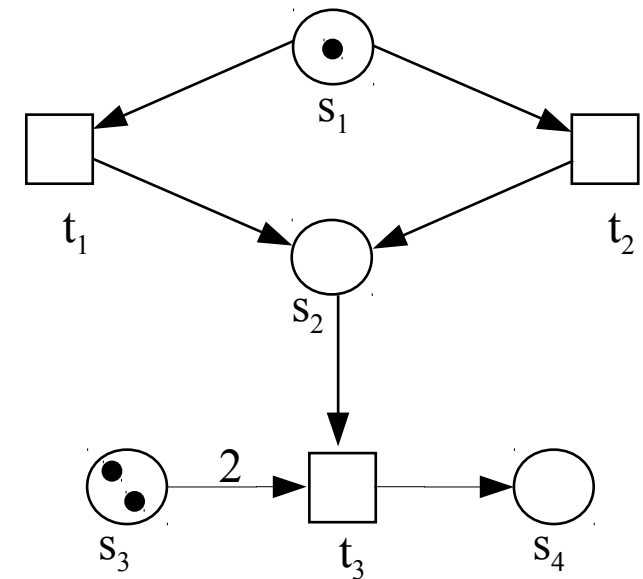
Des Weiteren:

- **Vorbereich einer Transition:** Menge aller Stellen, die über eine ausgehende Kante mit der Transition verbunden ist

Vorbereich von t : $\bullet t = \{s \in S \mid (s, t) \in F\}$

- **Nachbereich einer Transition:** Menge aller Stellen, die über eine eingehende Kante mit der Transition verbunden ist

Nachbereich von t : $t \bullet = \{s \in S \mid (t, s) \in F\}$

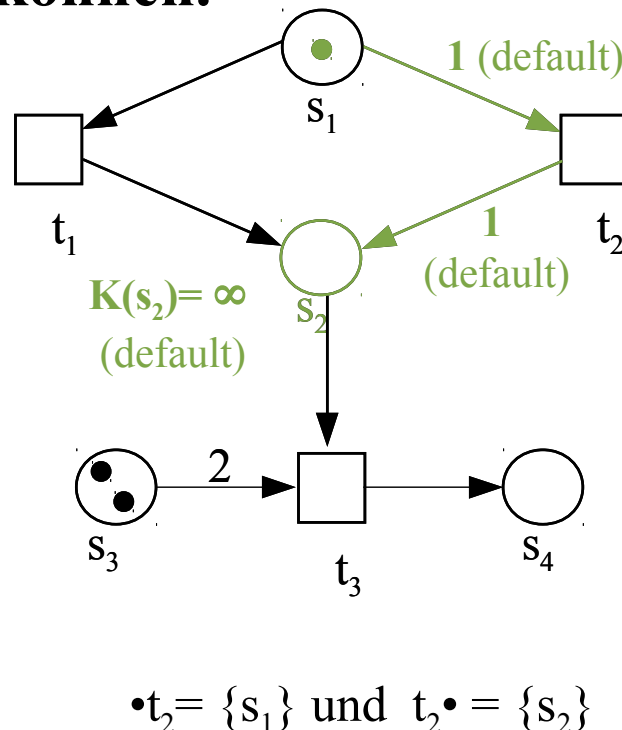


$\bullet t_1 = \bullet t_2 = \{s_1\}, \bullet t_3 = \{s_2, s_3\}$
 $t_1 \bullet = t_2 \bullet = \{s_2\}, t_3 \bullet = \{s_4\}$

- Transitionen sind **aktiviert**, wenn sie die
 - geforderte Anzahl Marken erhalten und
 - **Folgemarkierung** freigesetzte Marken **respektieren** (aufnehmen) **können**,
- d.h. wenn
 - alle Stellen im Vorbereich der Transition ausreichend Marken besitzen → gemäß Kantengewicht benötigte Marken liefern und
 - Kapazitäten aller Stellen im Nachbereich der Transition groß genug ist → gemäß Kanten freigesetzte Marken aufnehmen
- **Transition t ist aktiviert gdw.:**

$$\forall s \in \bullet t: M(s) \geq W(s, t) \wedge \forall s' \in t \bullet: M(s') + W(t, s') \leq K(s')$$

- Transitionen sind **aktiviert**, wenn sie die
 - geforderte Anzahl Marken erhalten und
 - **Folgemarkierung** freigesetzte Marken **respektieren** (aufnehmen) können.



Sei

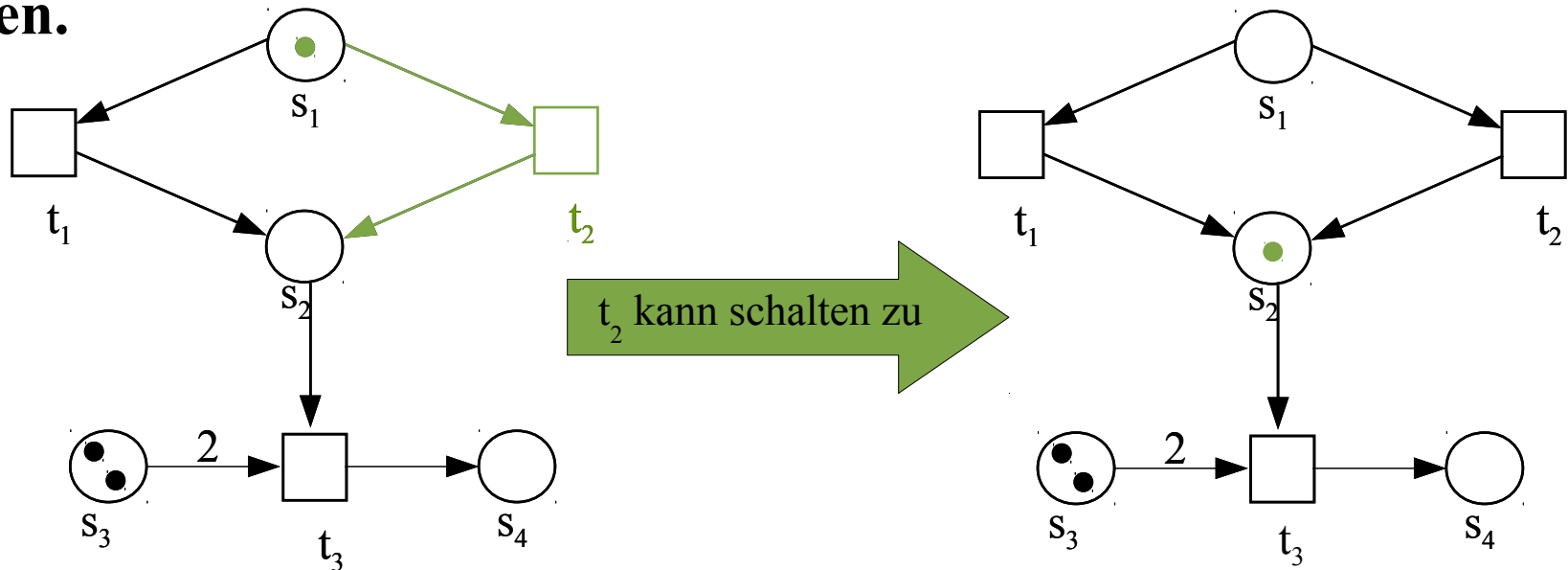
$$M_0(s_1)=1, M_0(s_3)=2, M_0(s_2)=M_0(s_4)=0$$

Die Aktivität t_2 ist **aktiviert**, da t_2 die benötigte(n) Marke(n) der Kante (t_2, s_2) von s_1 erhalten kann und die Kapazität von s_2 ausreicht, um die Marke(n) der Kante (t_2, s_2) aufzunehmen.

$$\forall s \in \bullet t_2 : M(s) \geq W(s, t_2)$$

$$\forall s' \in t_2 \bullet : M(s') + W(t_2, s') \leq K(s')$$

- Transitionen sind **aktiviert**, wenn sie die
 - geforderte Anzahl Marken erhalten und
 - **Folgemarkierung** freigesetzte Marken **respektieren** (aufnehmen) können.



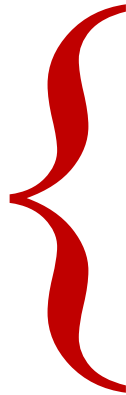
$$M_0(s_1) = 1, \quad M_0(s_2) = 0,$$

$$M_0(s_3) = 2, \quad M_0(s_4) = 0$$

$$M_1(s_1) = 0, \quad M_1(s_2) = 1,$$

$$M_1(s_3) = 2, \quad M_1(s_4) = 0$$

1.4 Petrinetze



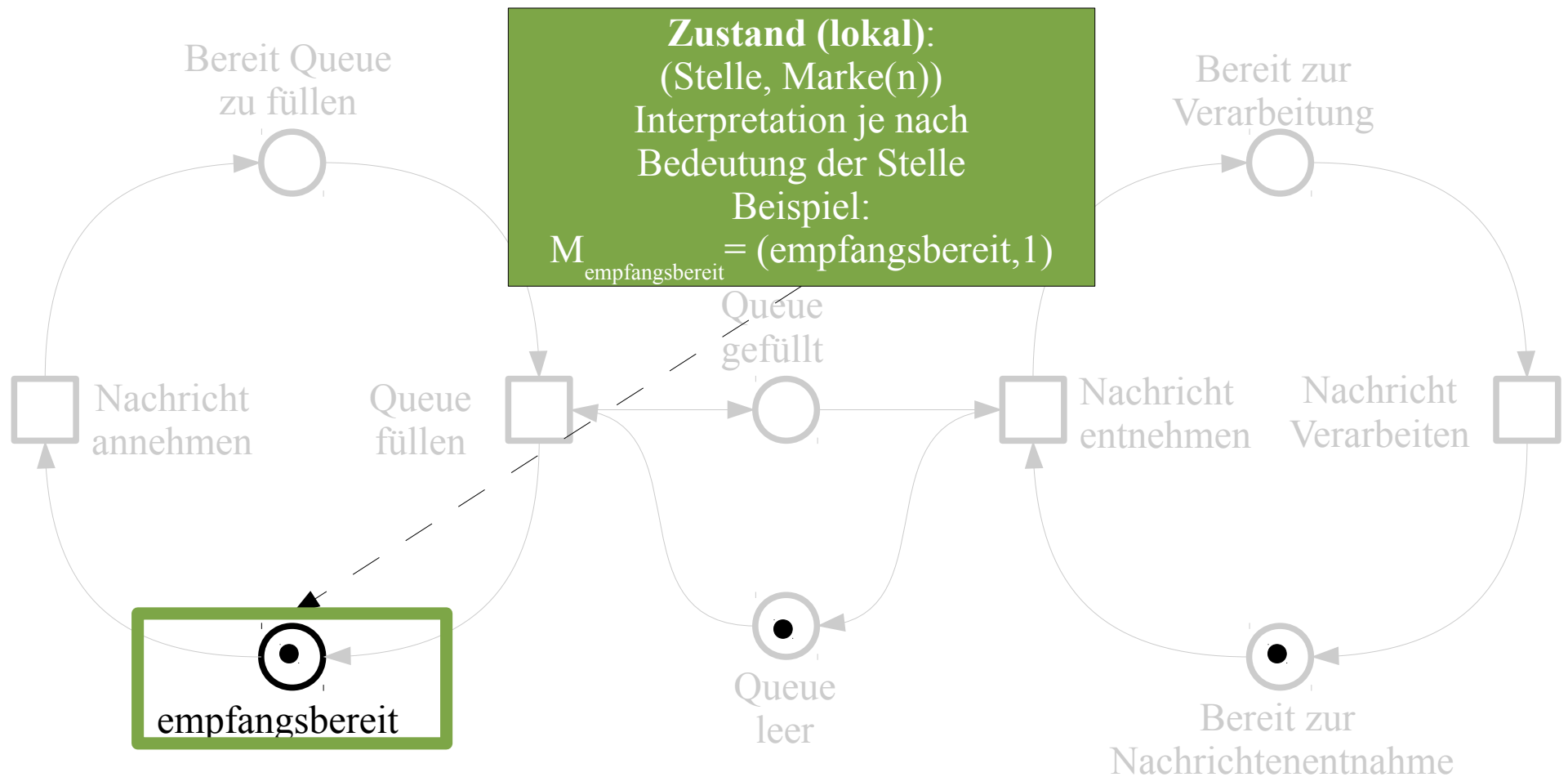
Petrinetz

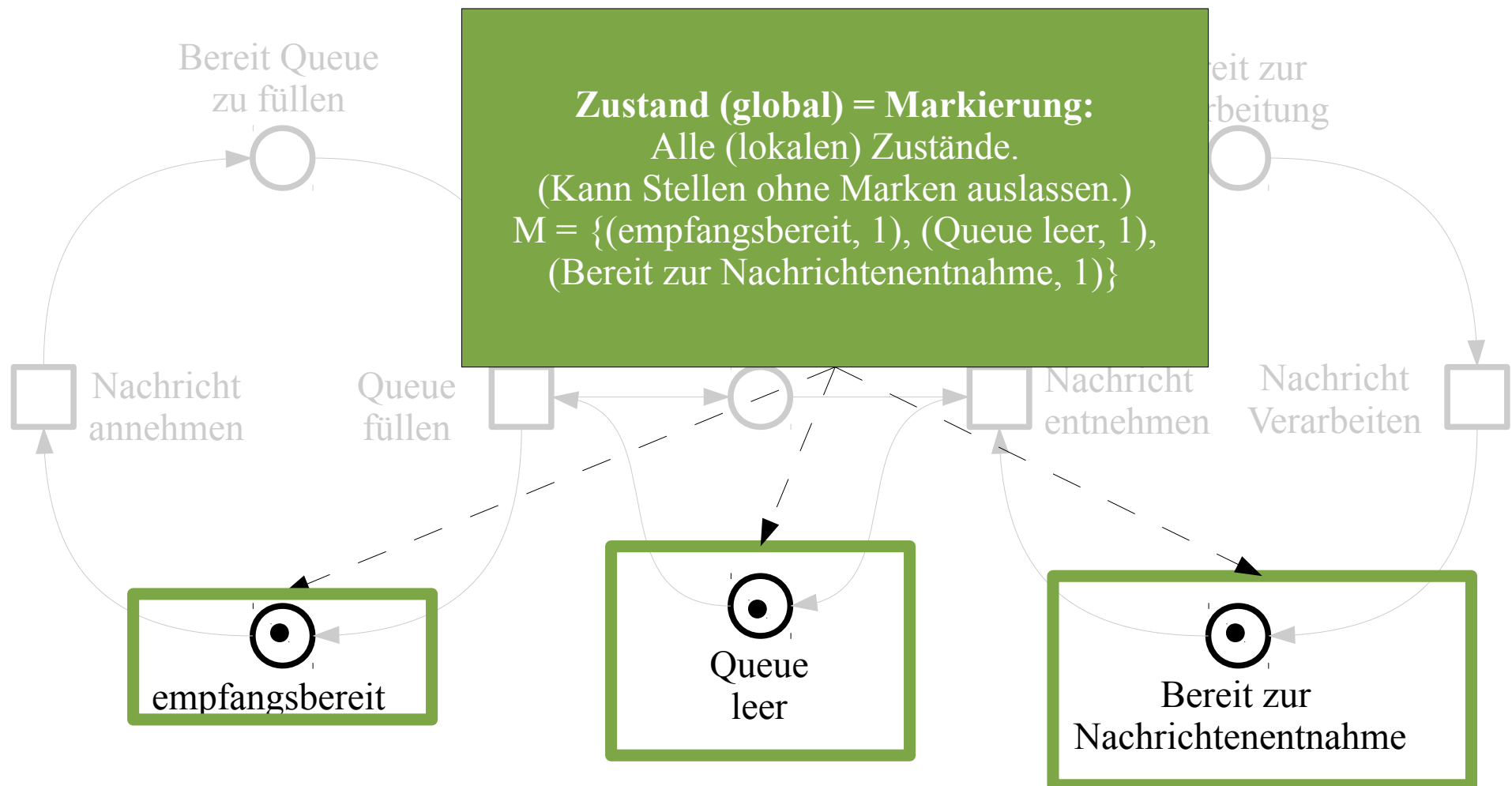
Ausführung

Analyse von Systemen

Workflow-Netze

Petrinetz-Ausführung: Zustand (lokal)





Markierung: Verteilung Marken auf Stellen (aktueller Systemzustand).

Initiale Markierung: Anfangszustand eines Netzes.

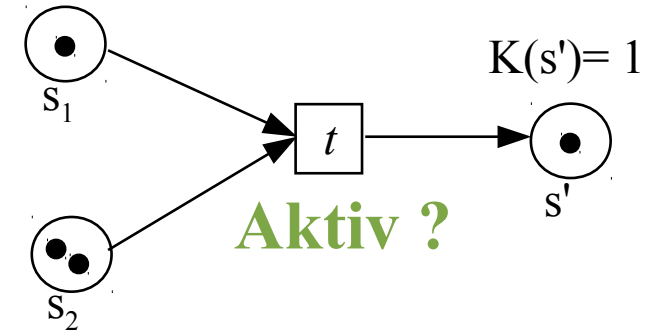
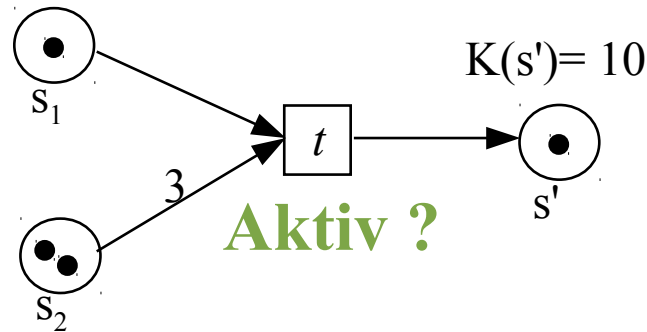
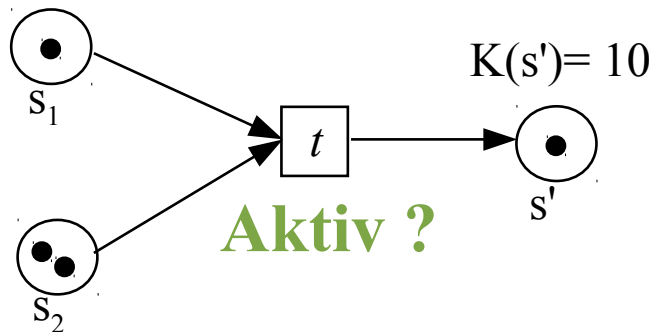
Verhaltenssimulation: evolvierende Anzahl Marken pro Stelle beobachten.

- Basierend auf aktueller Markierung: **aktivierte Transitionen** ermitteln.
Schalten führt zu Folgemarkierung.
- Unter Folgemarkierung sind (möglicherweise) andere Transitionen aktiviert.
- Solange iterieren, bis keine Transition mehr aktiv
($\Rightarrow$ „**tote Markierung**“).

Transition t ist aktiviert, wenn:

$$\forall s \in \text{Vorgänger von } t : M(s) \geq W(s, t) \text{ und} \\ \forall s' \in \text{Nachfolger von } t : M(s') + W(t, s') \leq K(s')$$

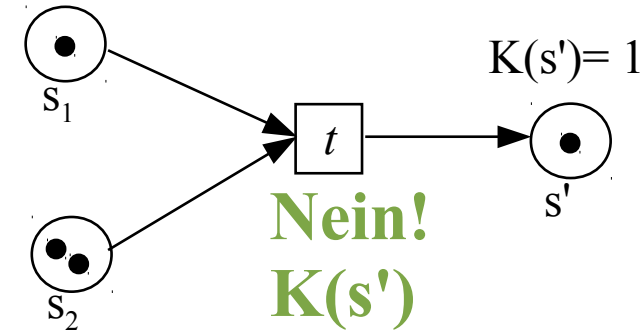
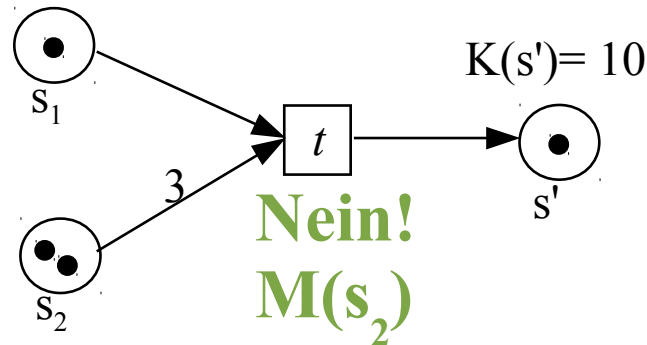
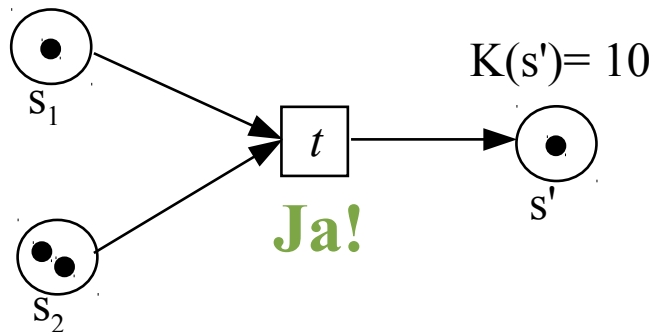
- $W(s, t)$: Gewicht des Bogens von s nach t
- $M(s)$: Anzahl Marken in s
- $K(s)$: Kapazität von s
- $W(t, s')$: Gewicht des Bogens von t nach s'
- $M(s')$: Anzahl Marken in s'
- $K(s')$: Kapazität von s'



Transition t ist aktiviert, wenn:

$$\forall s \in \text{Vorgänger von } t : M(s) \geq W(s, t) \text{ und} \\ \forall s' \in \text{Nachfolger von } t : M(s') + W(t, s') \leq K(s')$$

- $W(s, t)$: Gewicht des Bogens von s nach t
- $M(s)$: Anzahl Marken in s
- $K(s)$: Kapazität von s
- $W(t, s')$: Gewicht des Bogens von t nach s'
- $M(s')$: Anzahl Marken in s'
- $K(s')$: Kapazität von s'

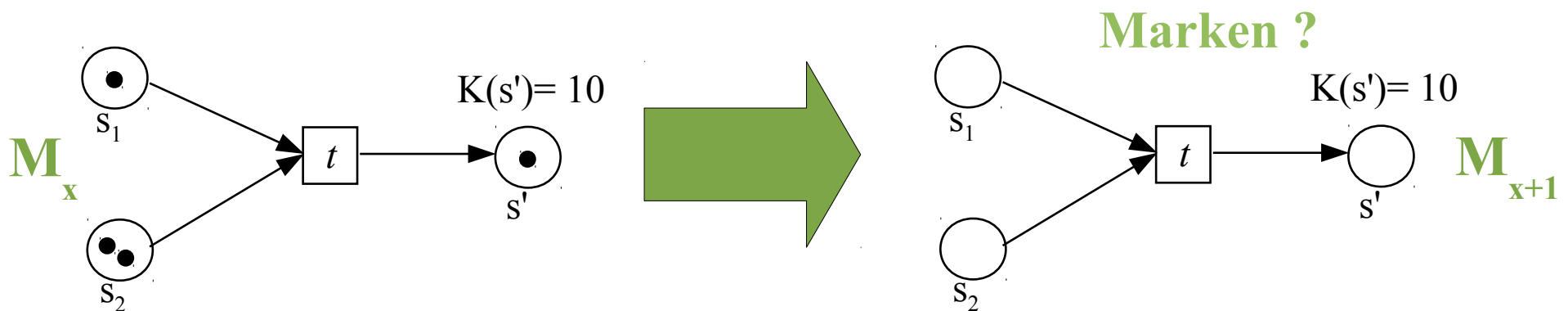


Eine der aktivierten Transitionen wird von Zustand M_x nach Zustand M_{x+1} geschaltet (**nicht-deterministische Auswahl**):

- Benötigte Marken auf **Vorgänger**-Stellen werden **konsumiert**.
- Produzierte Marken auf **Nachfolger**-Stellen abgelegt.

Anzahl konsumierter / produzierter Marken jeweils gemäß **Bogenvielfalt**:
→ **Gesamtanzahl** Marken im Netz kann sich **verändern**.

Folgemarkierung (= -zustand): Schalten jeweils **genau einer** Transition.

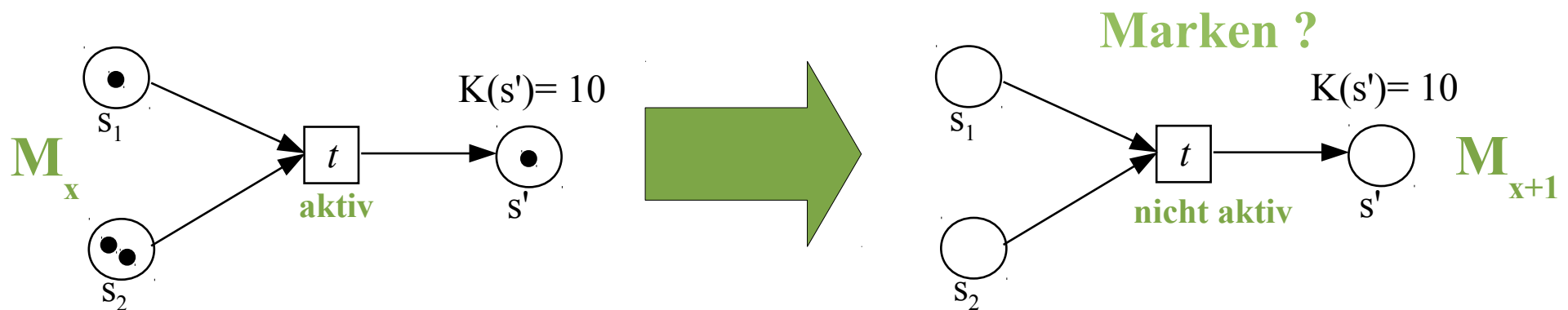


Eine der aktivierten Transitionen wird von Zustand M_x nach Zustand M_{x+1} geschaltet (**nicht-deterministische Auswahl**):

- Benötigte Marken auf **Vorgänger**-Stellen werden **konsumiert**.
- Produzierte Marken auf **Nachfolger**-Stellen abgelegt.

Anzahl konsumierter / produzierter Marken jeweils gemäß **Bogenvielfalt**:
→ **Gesamtanzahl** Marken im Netz kann sich **verändern**.

Folgemarkierung (= -zustand): Schalten jeweils **genau einer** Transition.

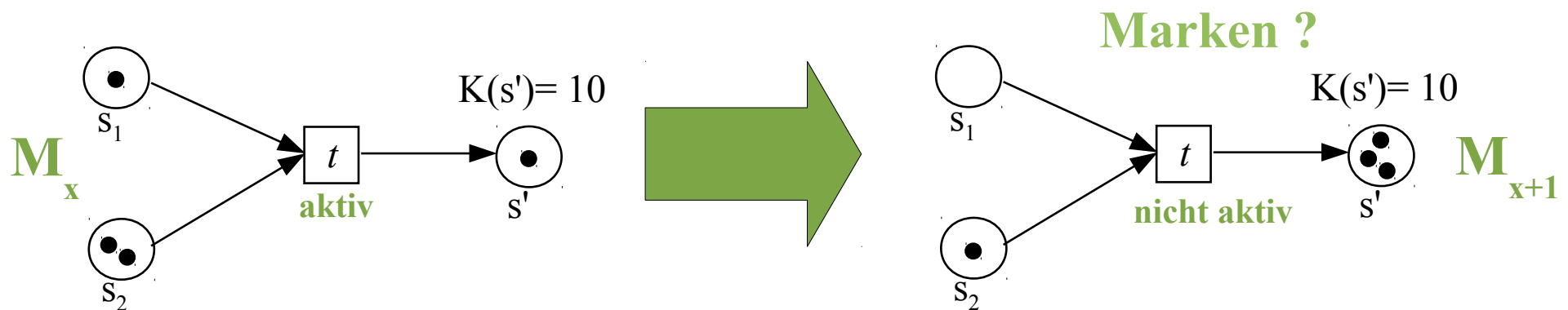


Eine der aktivierten Transitionen wird von Zustand M_x nach Zustand M_{x+1} geschaltet (**nicht-deterministische Auswahl**):

- Benötigte Marken auf **Vorgänger**-Stellen werden **konsumiert**.
- Produzierte Marken auf **Nachfolger**-Stellen abgelegt.

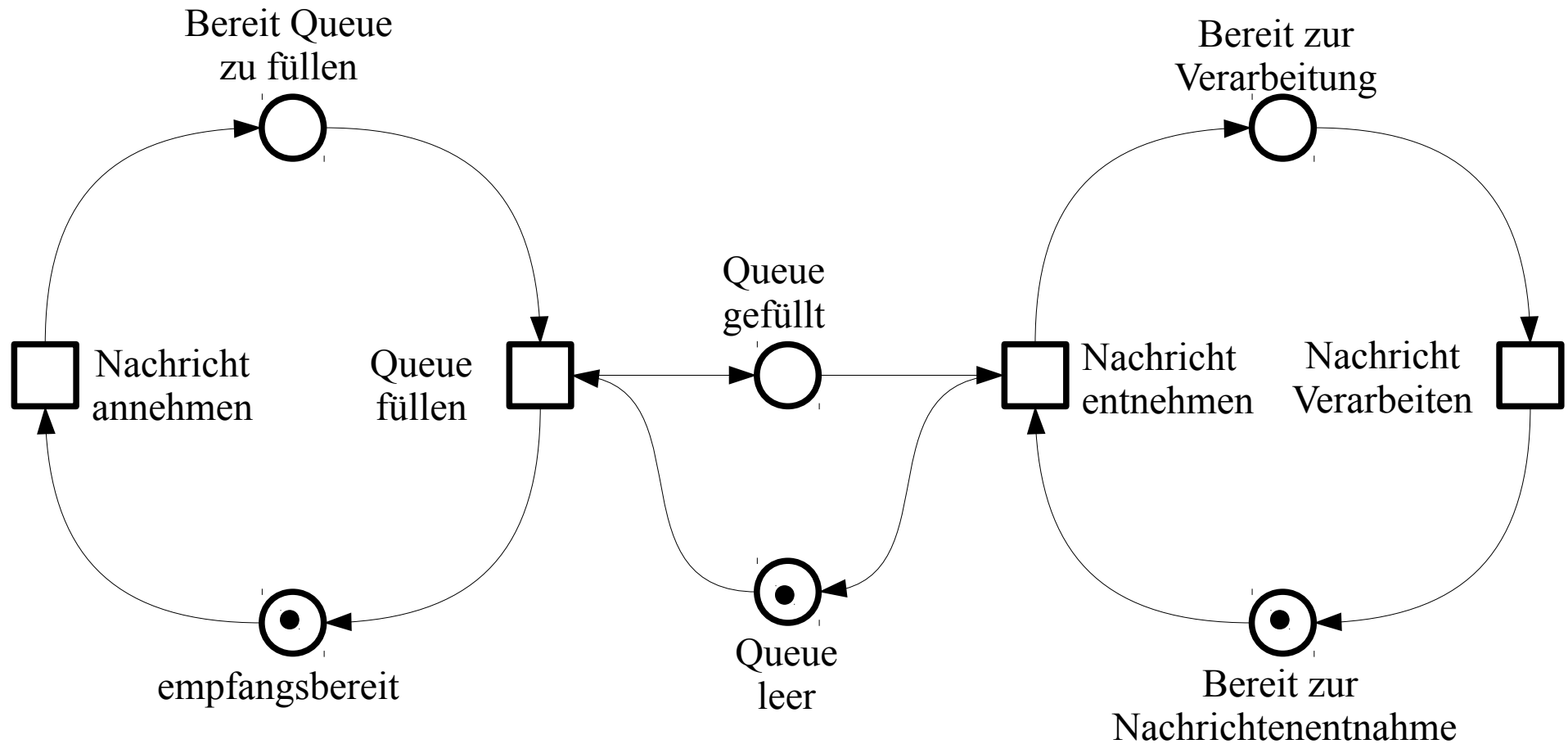
Anzahl konsumierter / produzierter Marken jeweils gemäß **Bogenvielfalt**:
→ **Gesamtanzahl** Marken im Netz kann sich **verändern**.

Folgemarkierung (= -zustand): Schalten jeweils **genau einer** Transition.

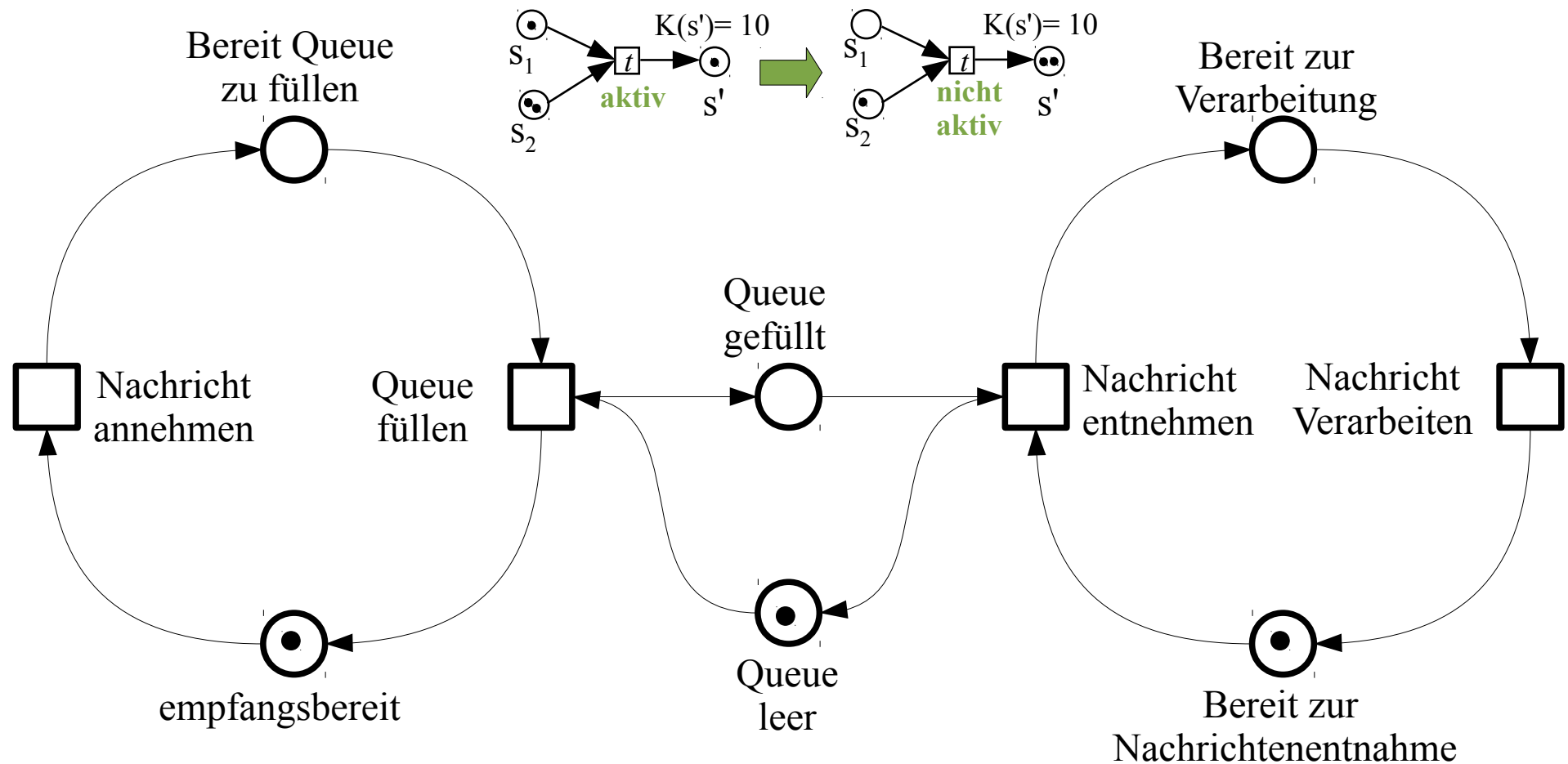


Petrinetz Ablauf: Beispiel Nachrichten-Queue

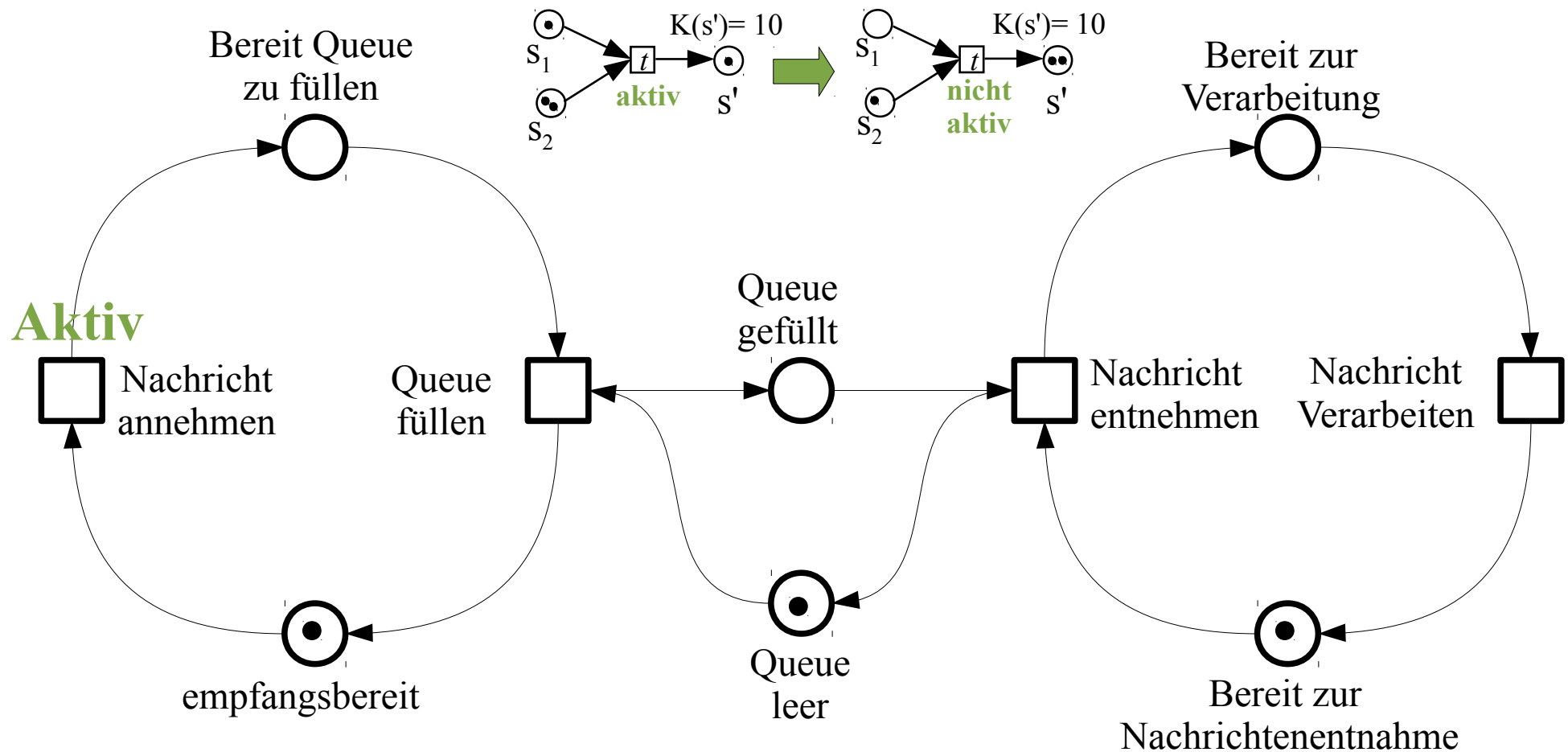
Softwarekonstruktion
WS 2016/17



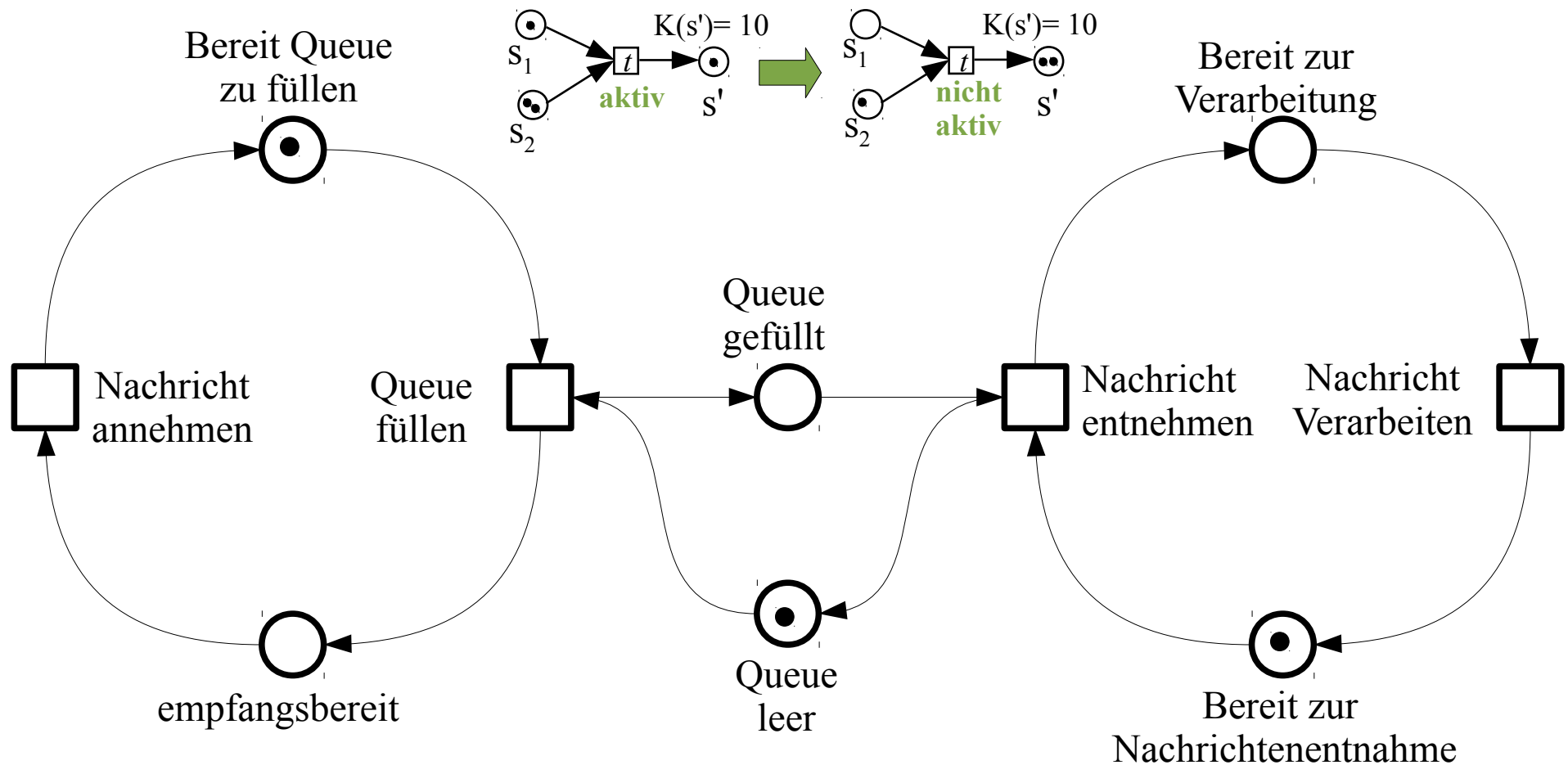
Welche Transition(en) aktiviert ?



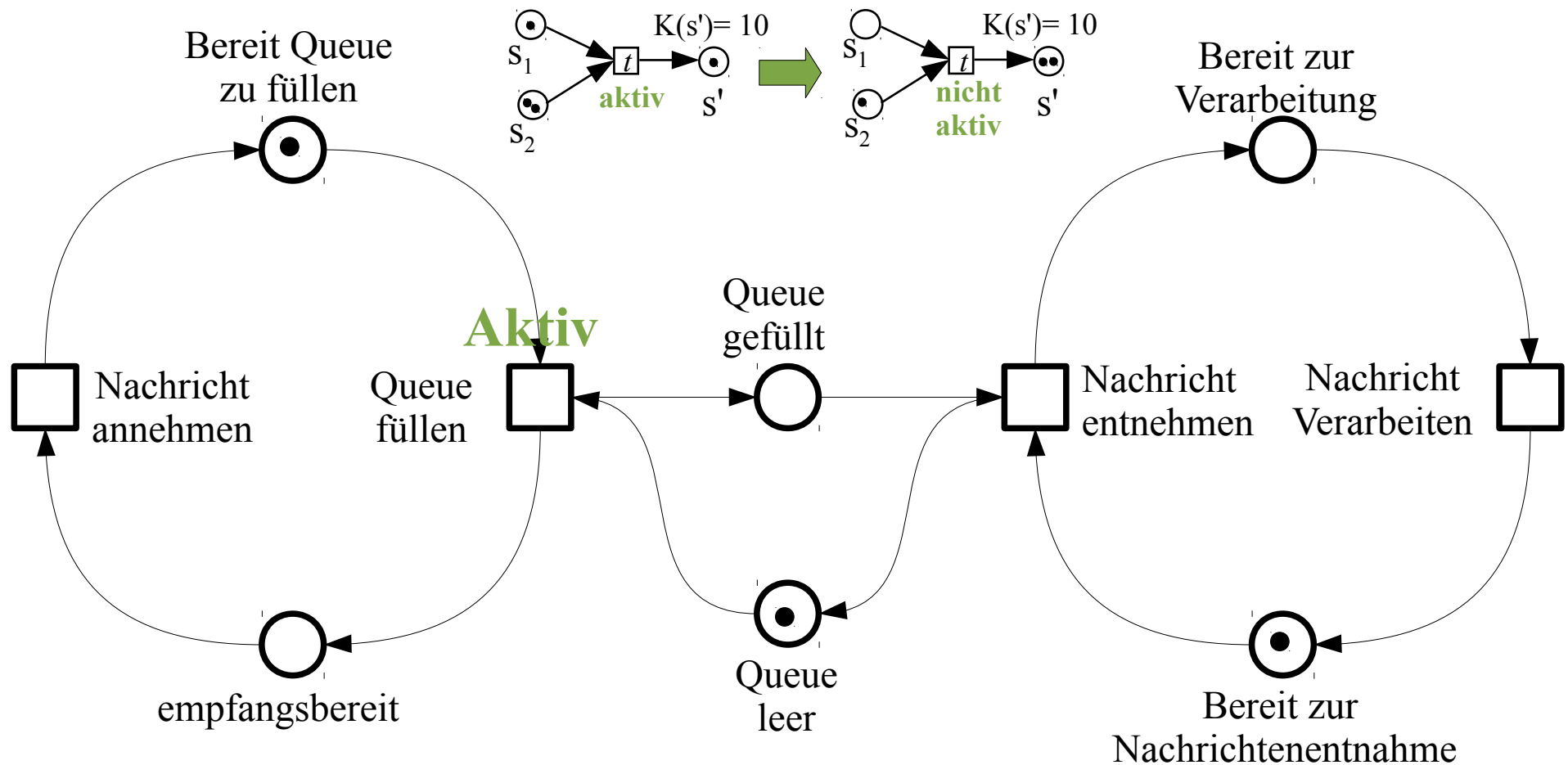
Nächster Zustand ?



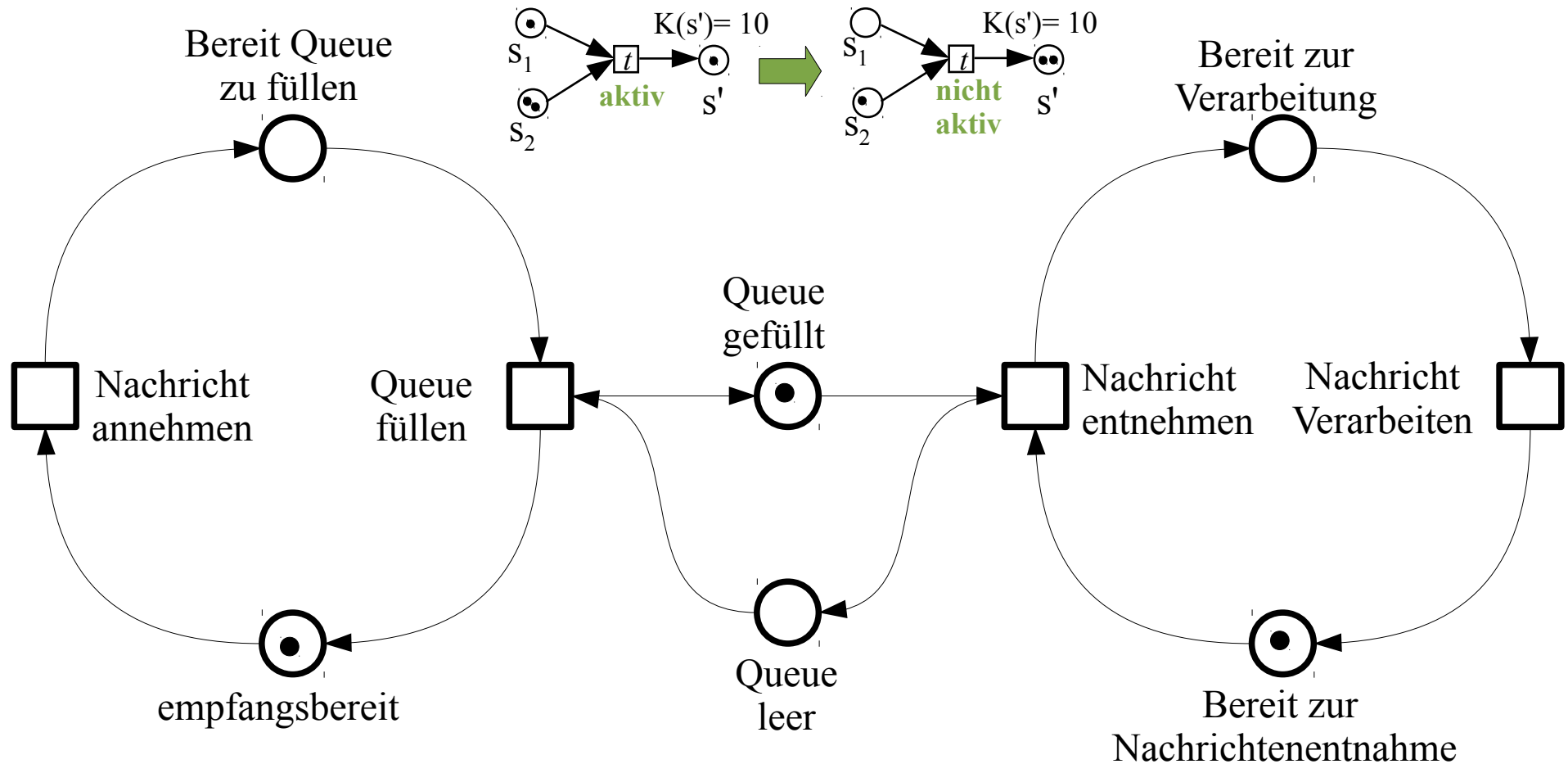
Welche Transition(en) aktiviert ?



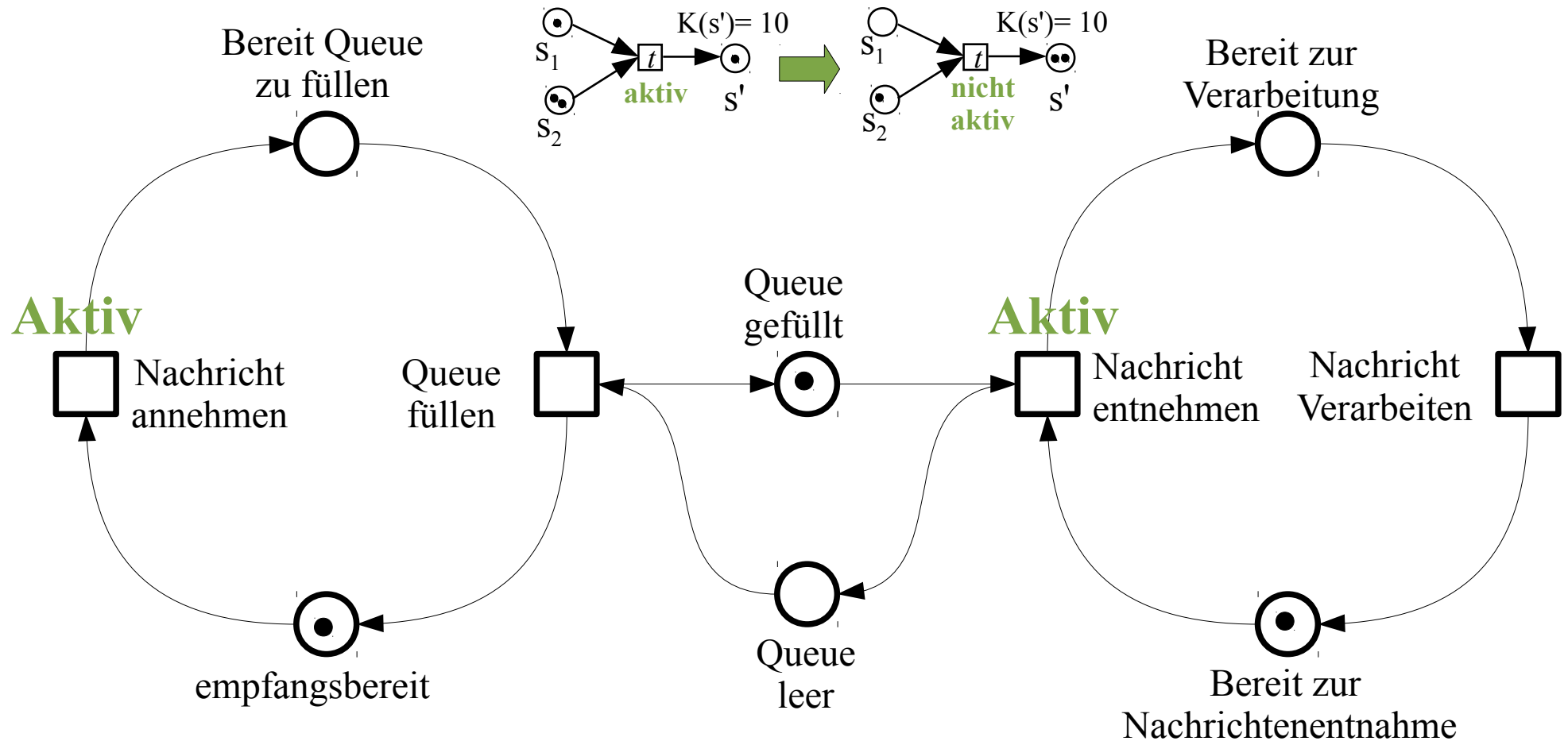
Nächster Zustand ?

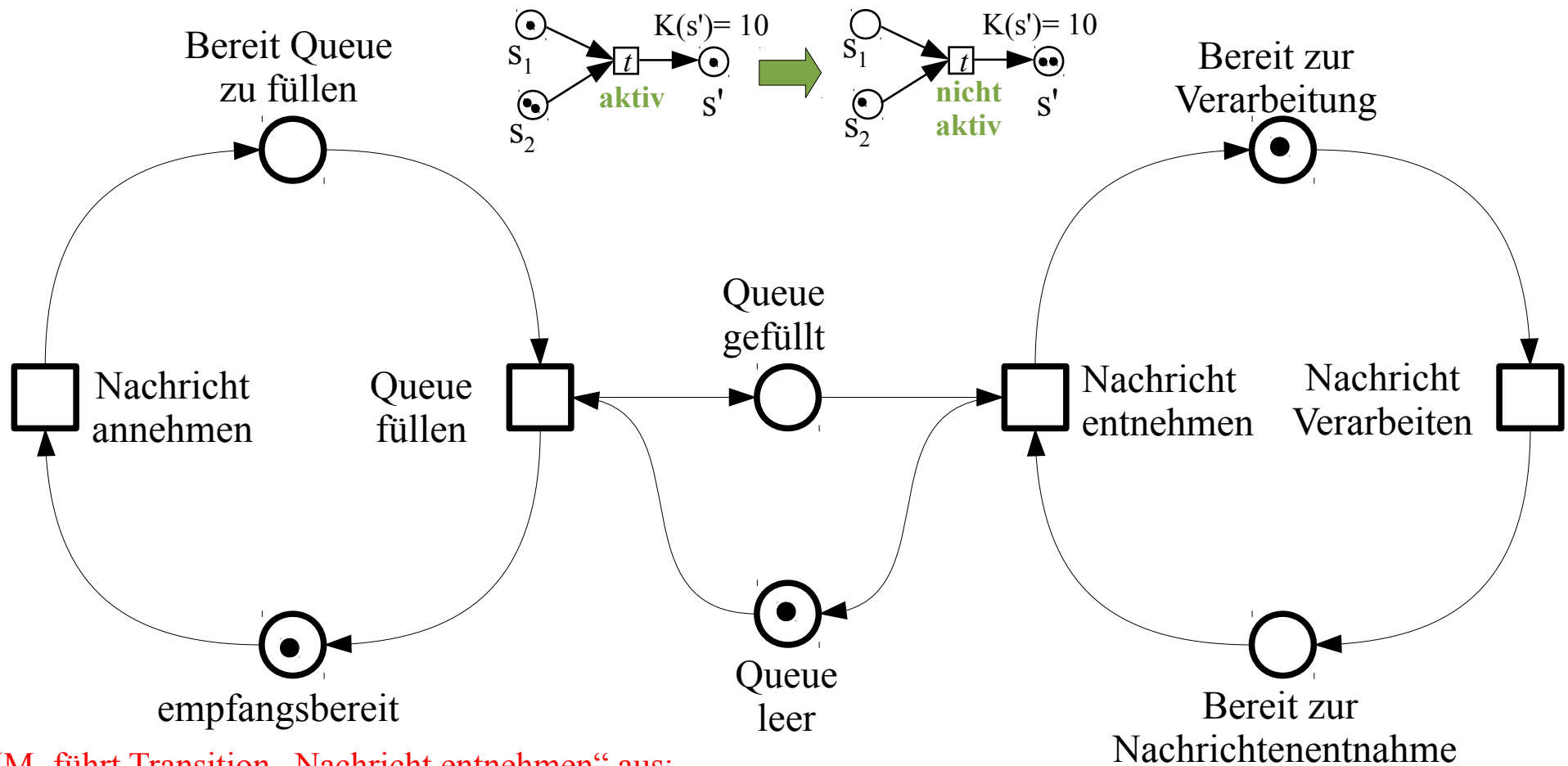


Welche Transition(en) aktiviert ?



Nächster Zustand ?





(M_2 führt Transition „Nachricht entnehmen“ aus;
alternativ: „Nachricht annehmen“ ausführbar.)

Gibt es eine obere Grenze, wieviele Nachrichten gleichzeitig in dieser Queue
enthalten sein können ?

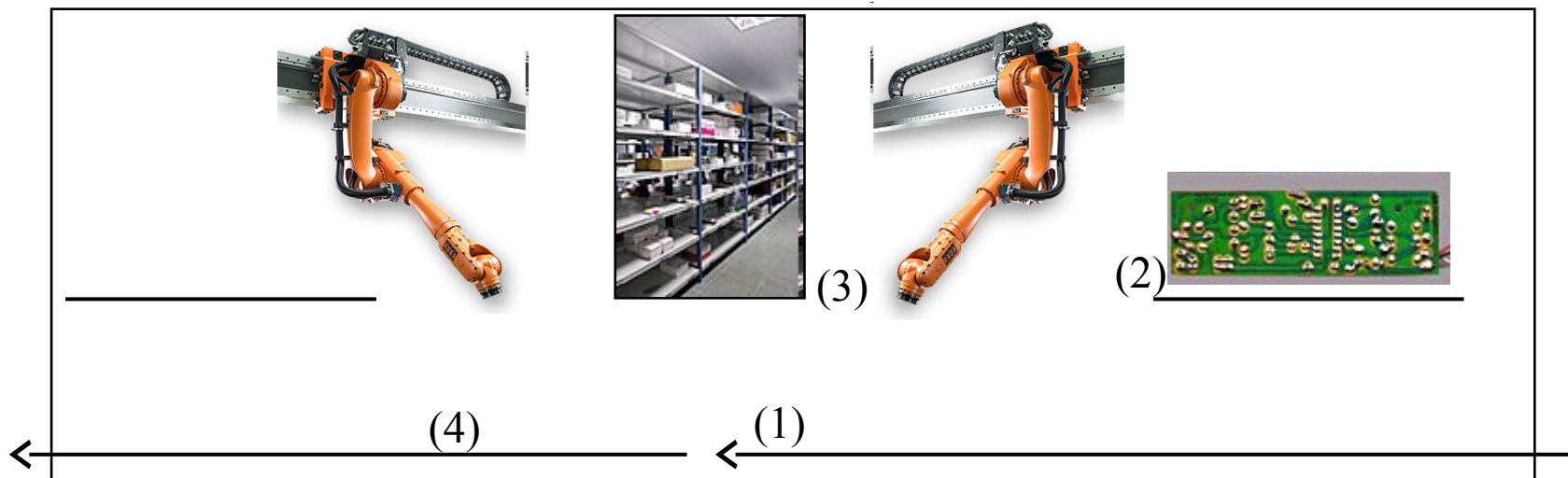
Gibt es eine obere Grenze, wieviele Nachrichten gleichzeitig in dieser Queue enthalten sein können ?

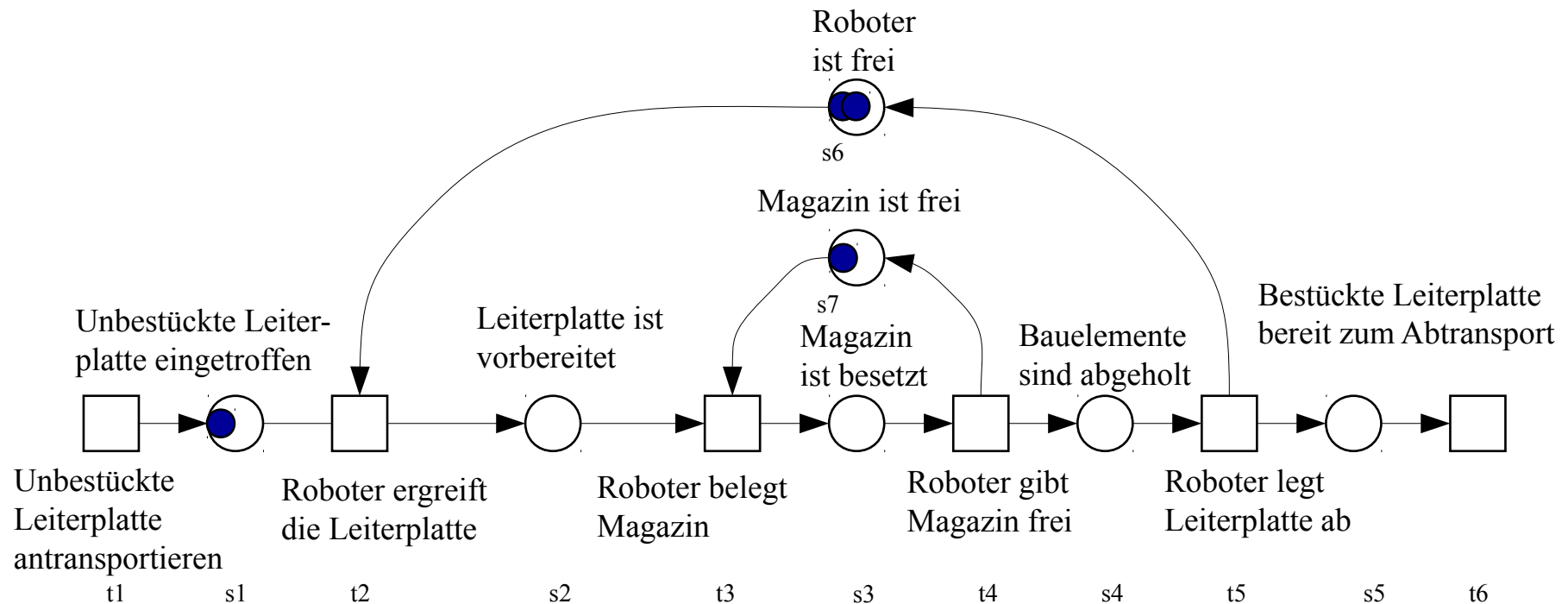
Antwort:

- In der Queue: höchstens **eine** Nachricht .
- Stellen „Nachricht empfangen“ und „Queue leer“ besitzen einen Marker. → Transition „Queue füllen“ ausführen.

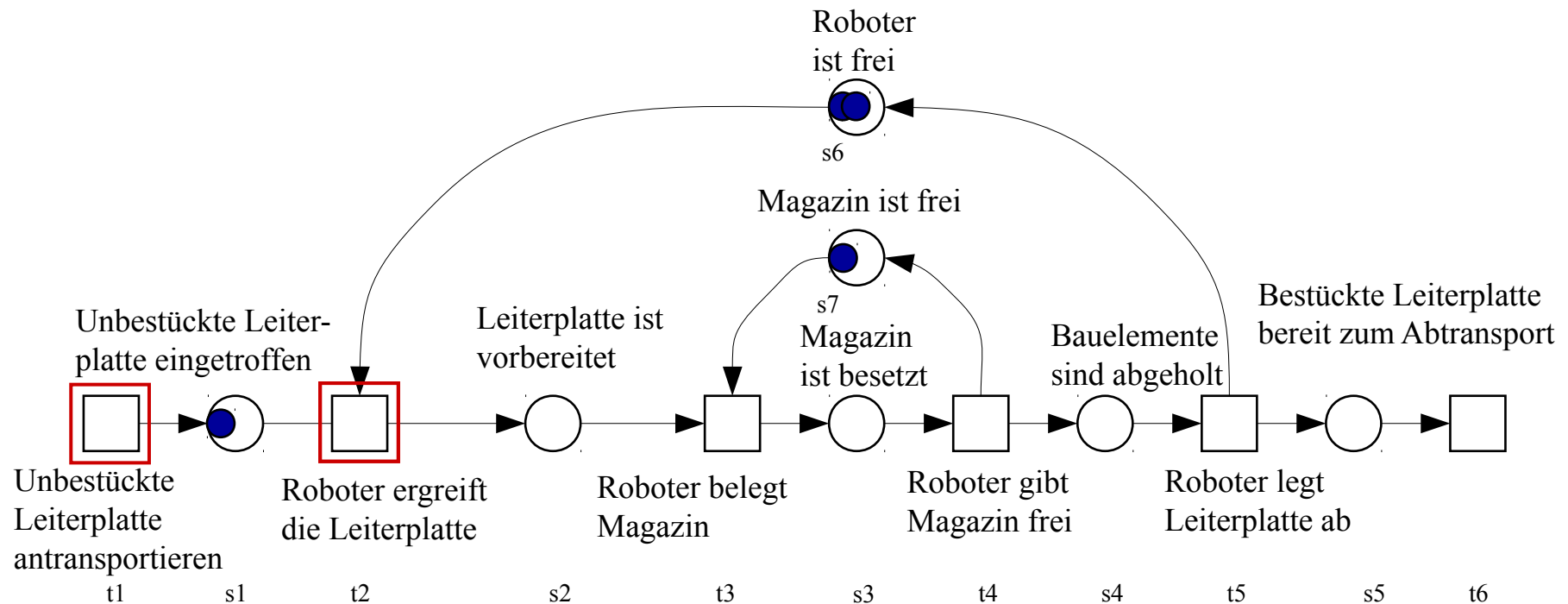
Zwei Roboter bestücken Leiterplatten mit elektronischen Bauelementen:

- Leiterplatten auf Fließband antransportiert (1).
- Freier Roboter nimmt Leiterplatte vom Fließband (2).
- Beide Roboter frei: nichtdeterministisch entschieden, wer Leiterplatte nimmt.
- Jeweils ein Roboter darf auf Bauelemente-Magazin zugreifen (3), um Leiterplatte mit Bauelementen zu bestücken.
- Jeweils eine Leiterplatte zu einem Zeitpunkt abtransportierbar (4).

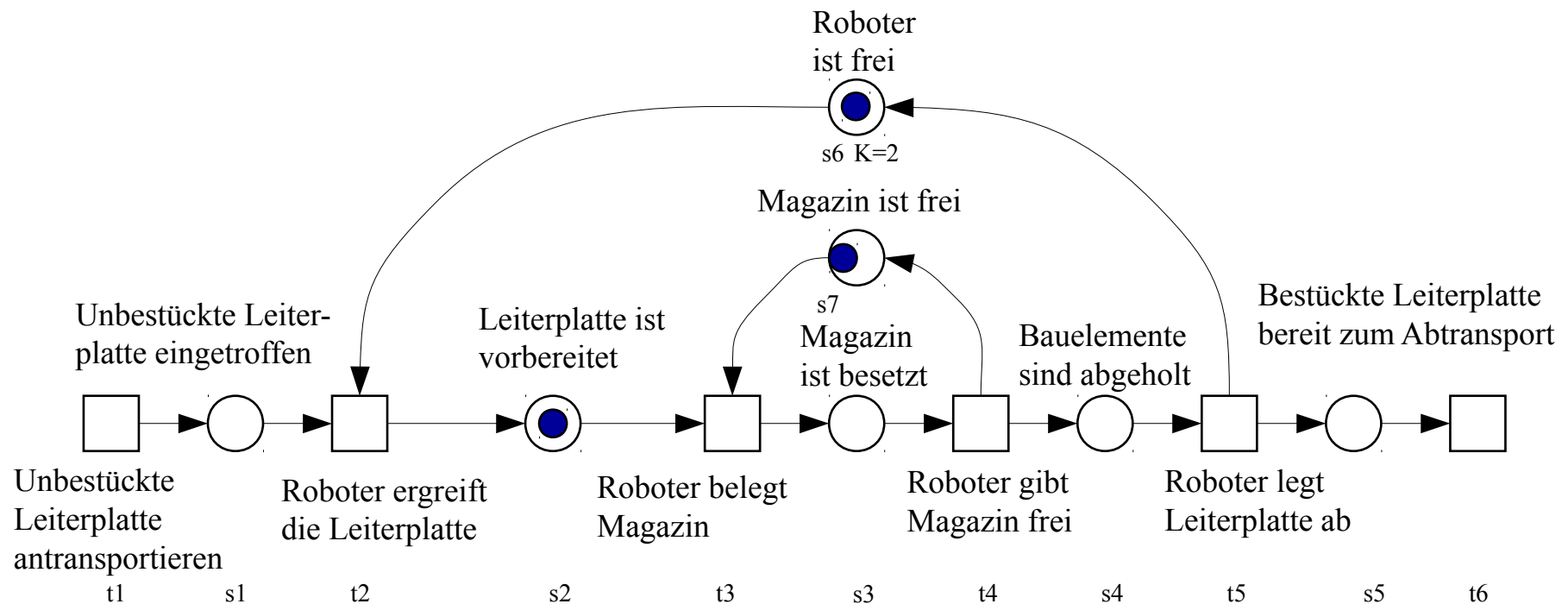




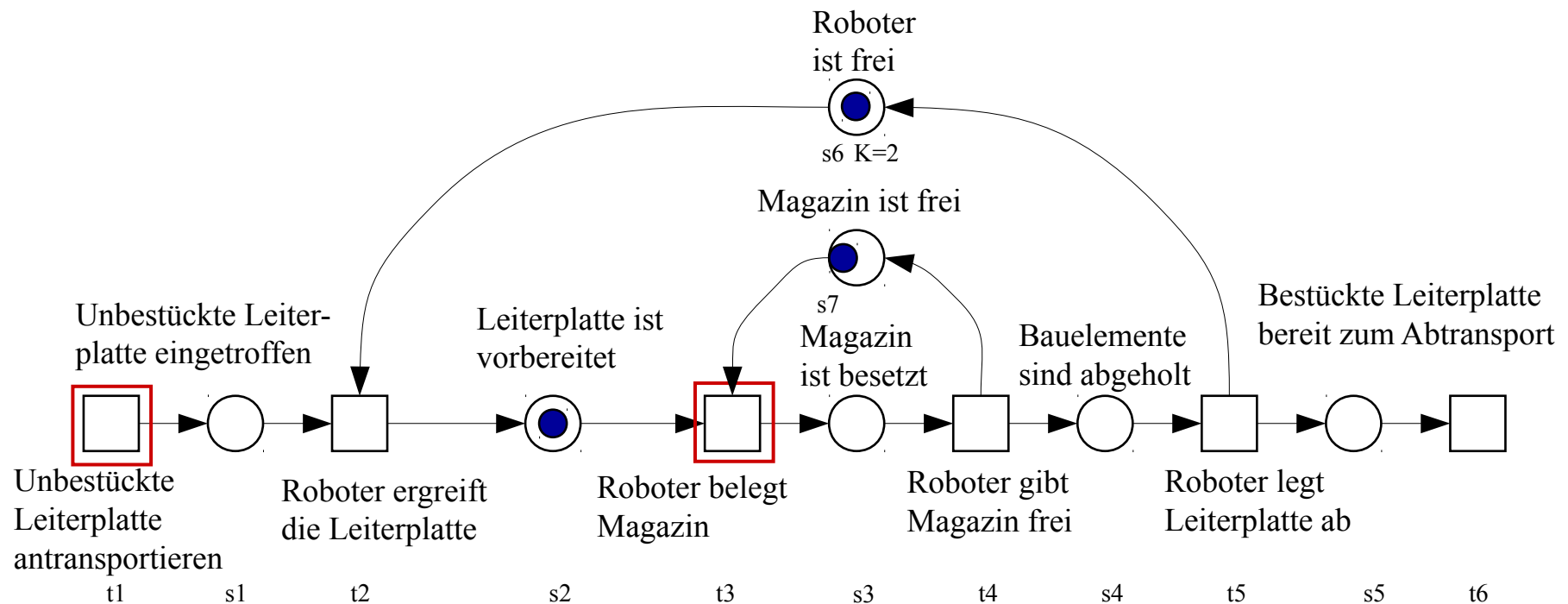
Welche Transition(en) aktiviert ?



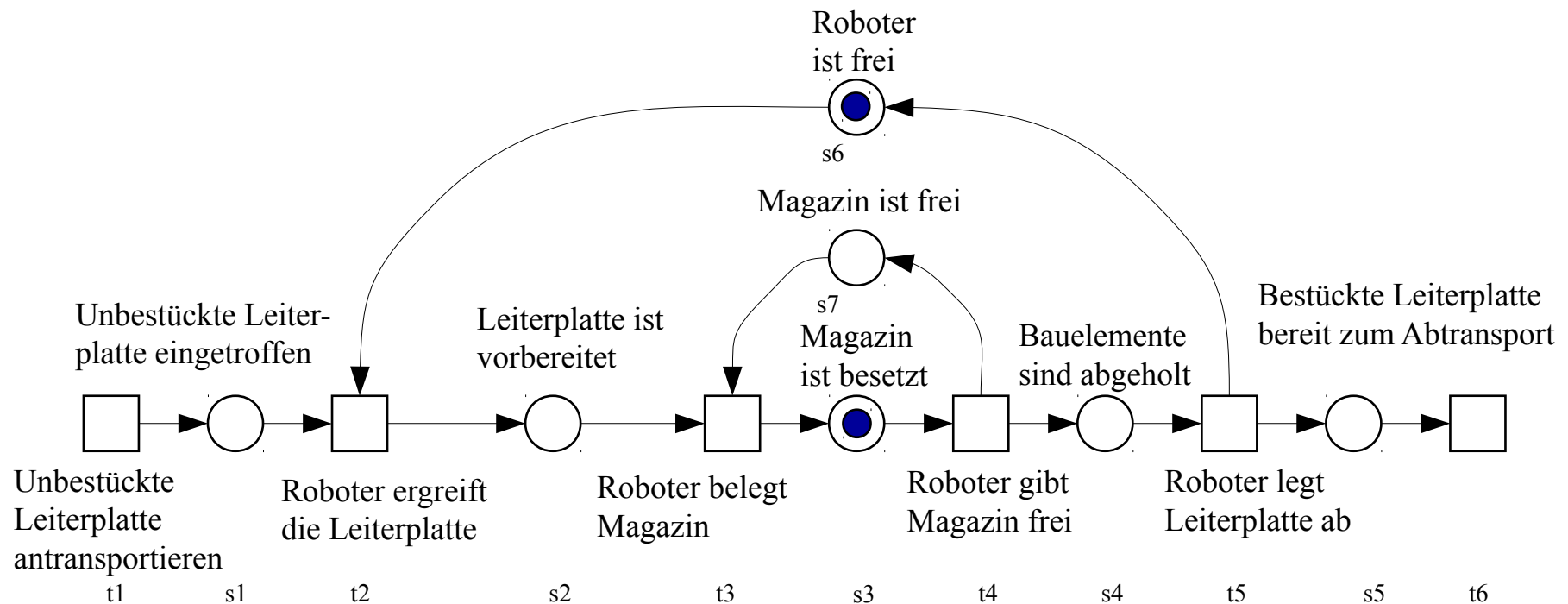
 bezeichnet aktivierte Transitionen. Möglicher nächster Zustand ?



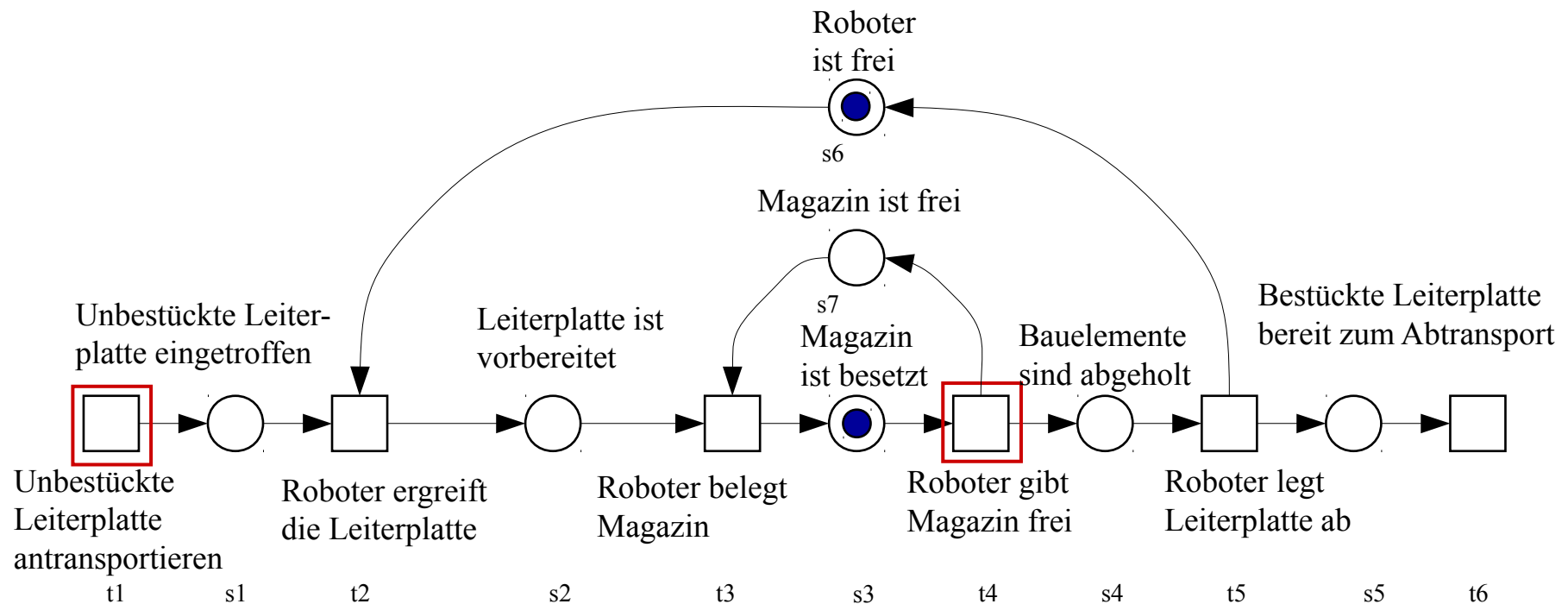
Welche Transition(en) aktiviert ?



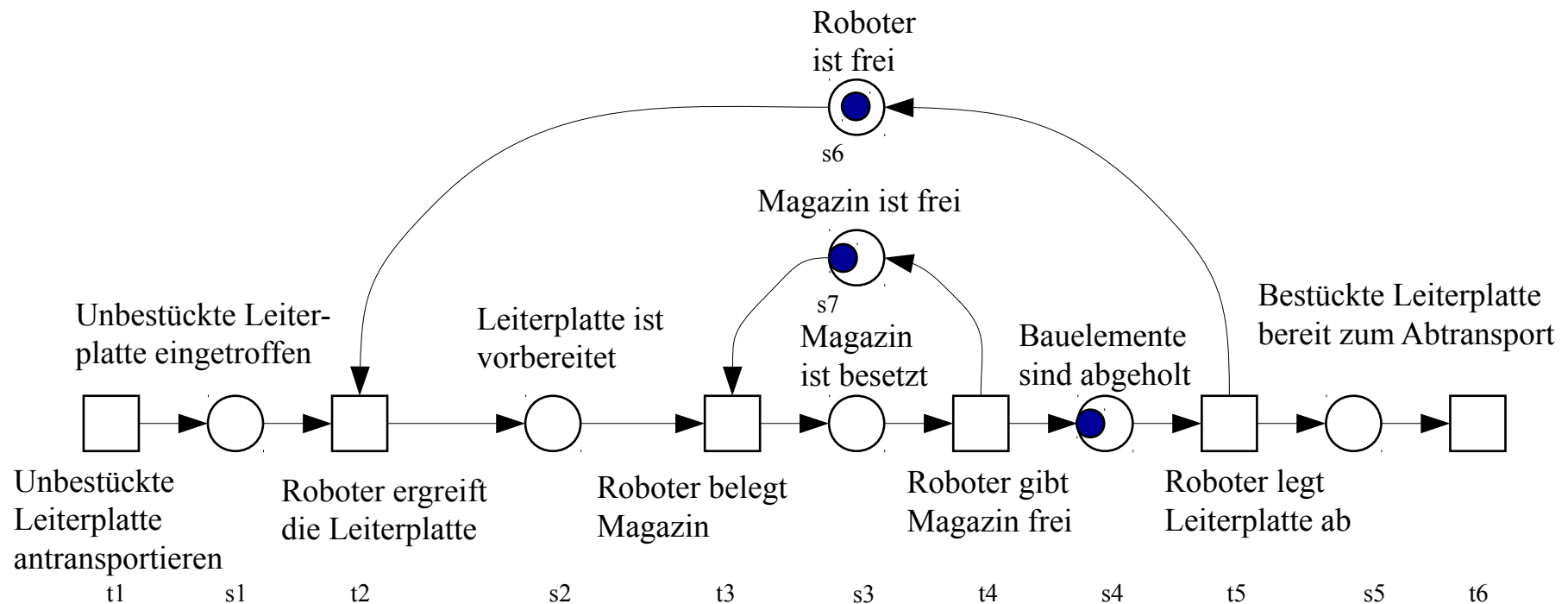
Möglicher nächster Zustand ?



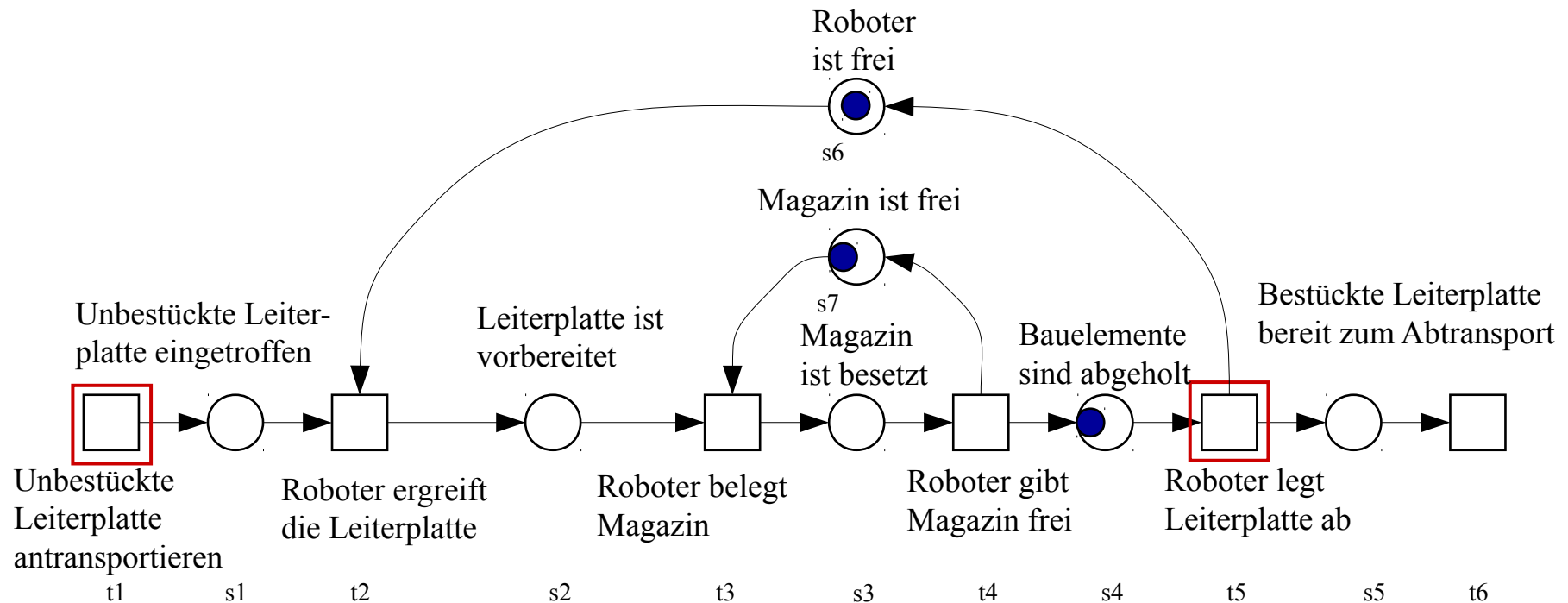
Welche Transition(en) aktiviert ?



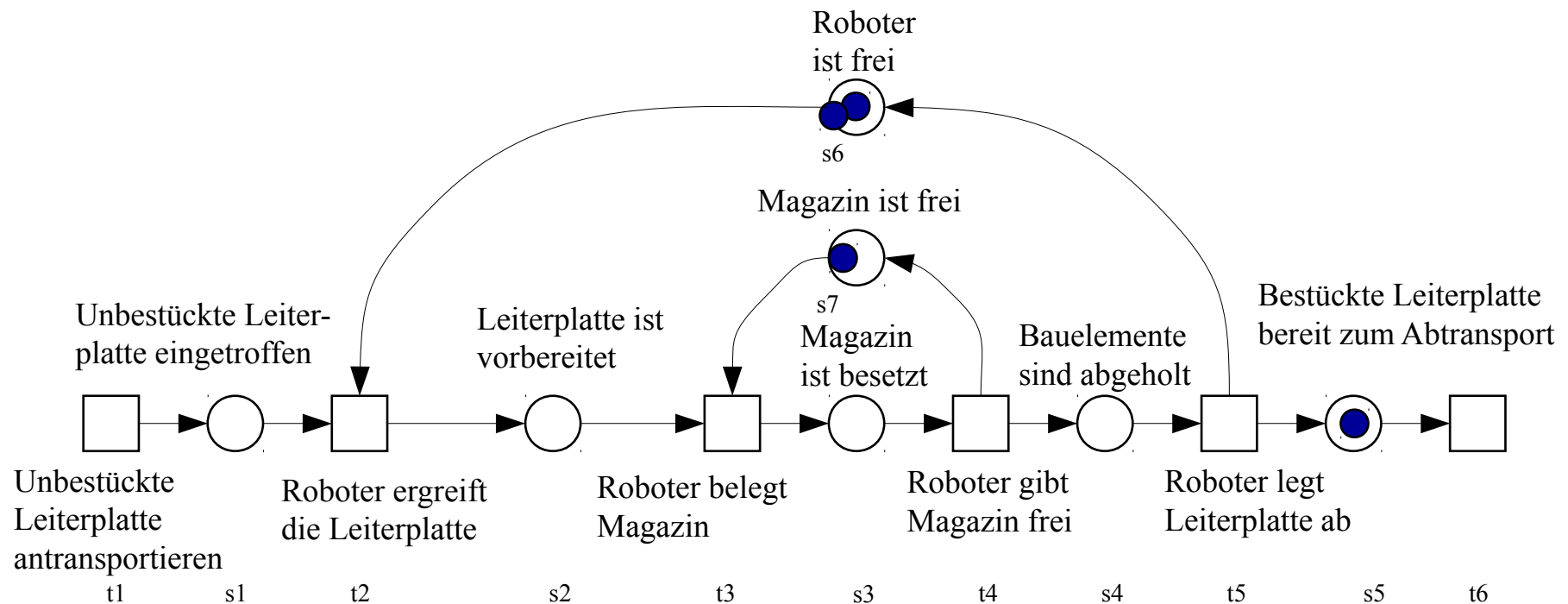
Möglicher nächster Zustand ?



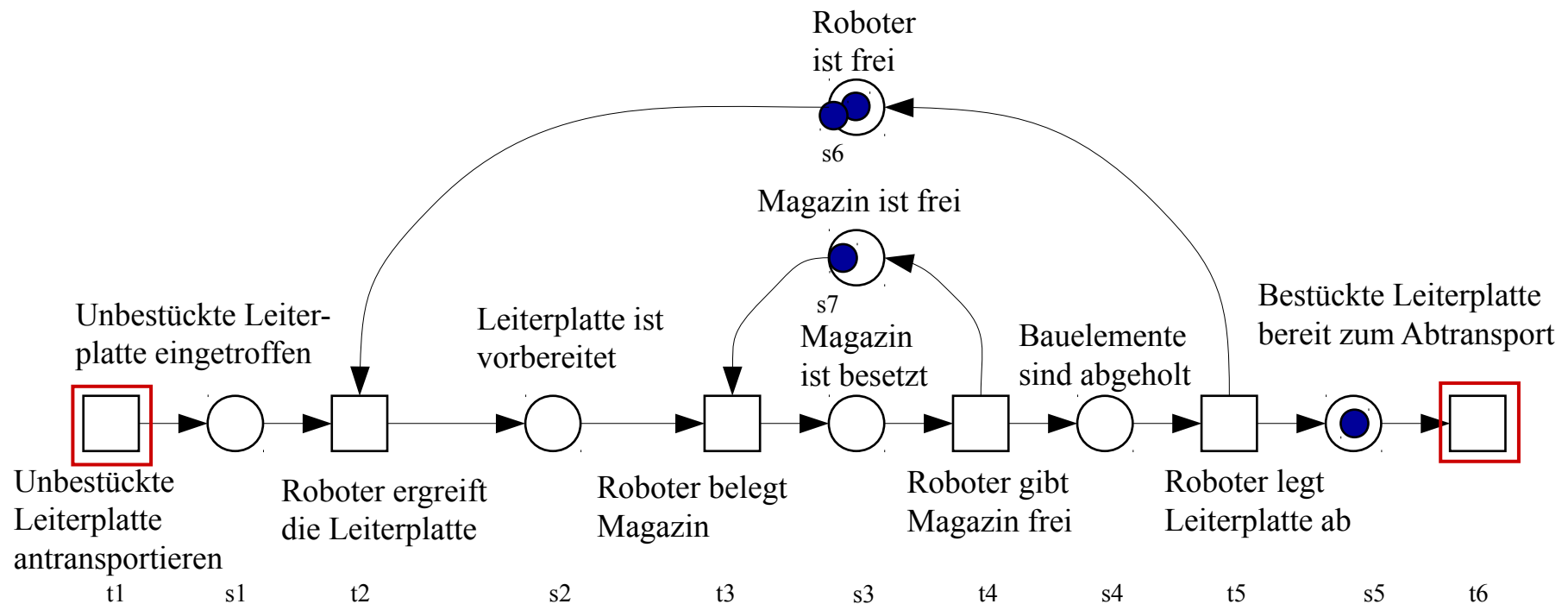
Welche Transition(en) aktiviert ?



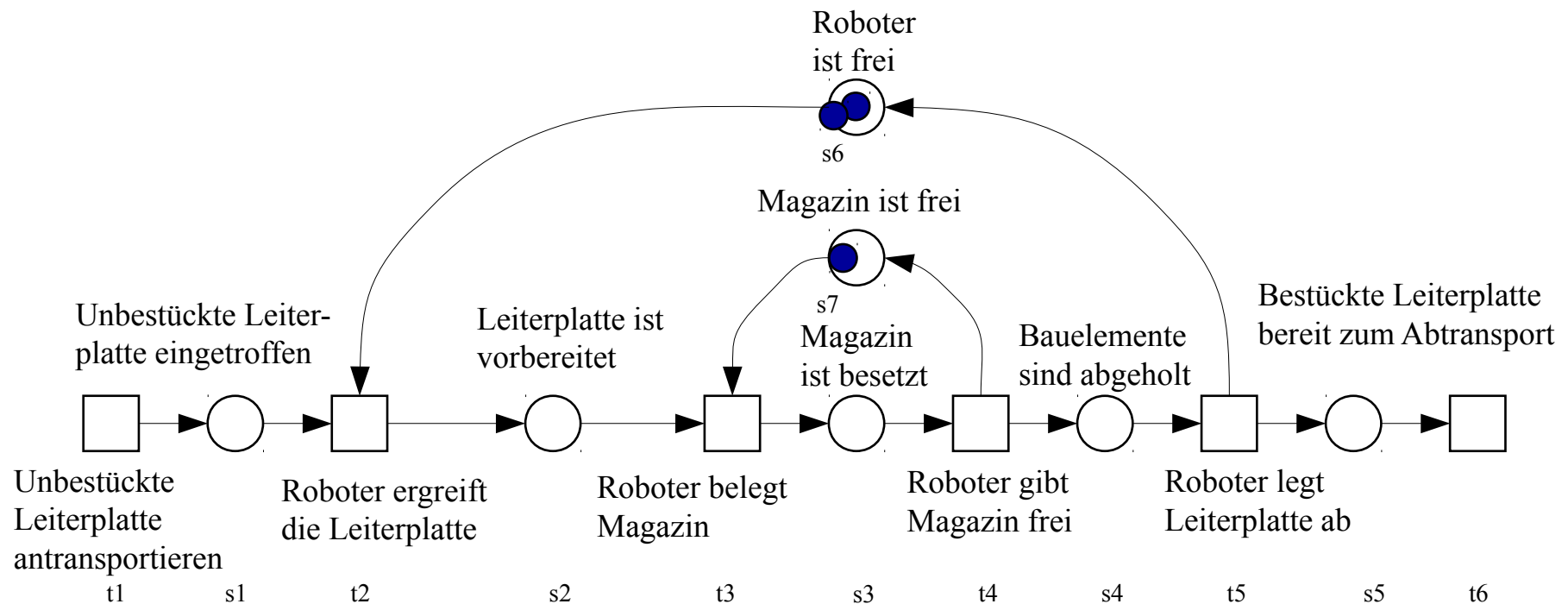
Möglicher nächster Zustand ?



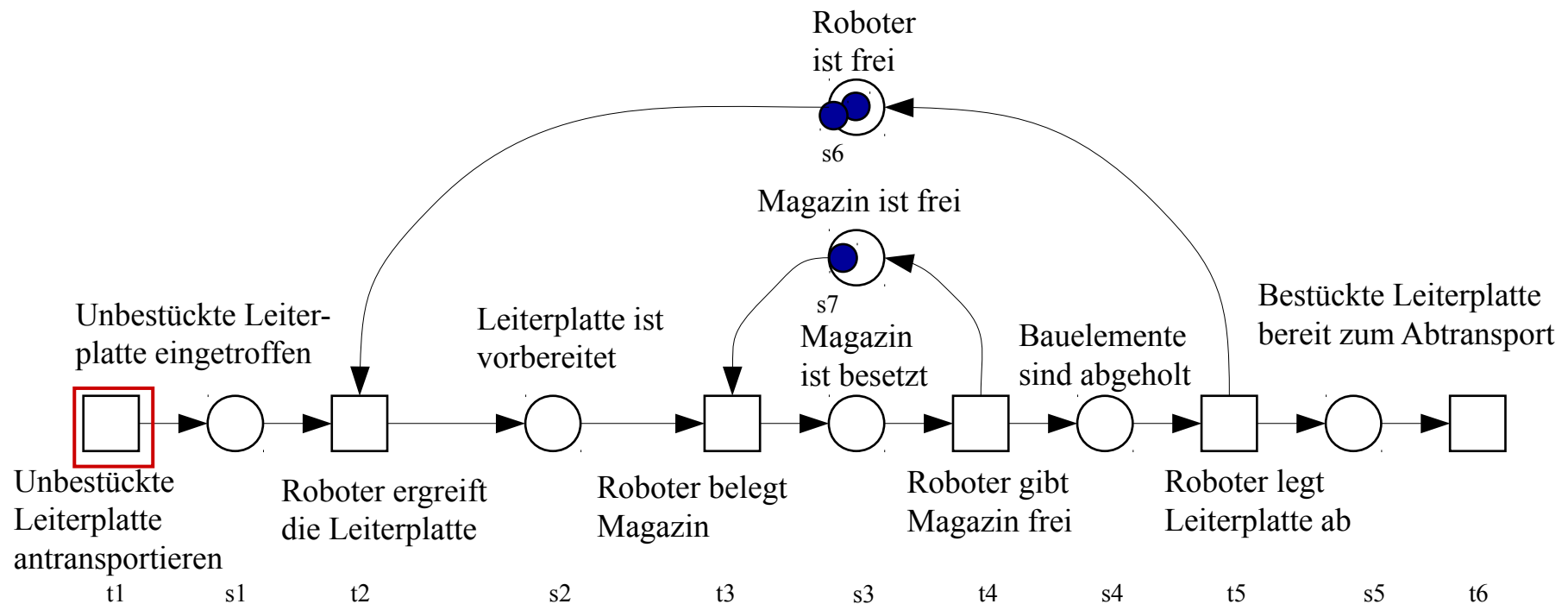
Welche Transition(en) aktiviert ?



Möglicher nächster Zustand ?



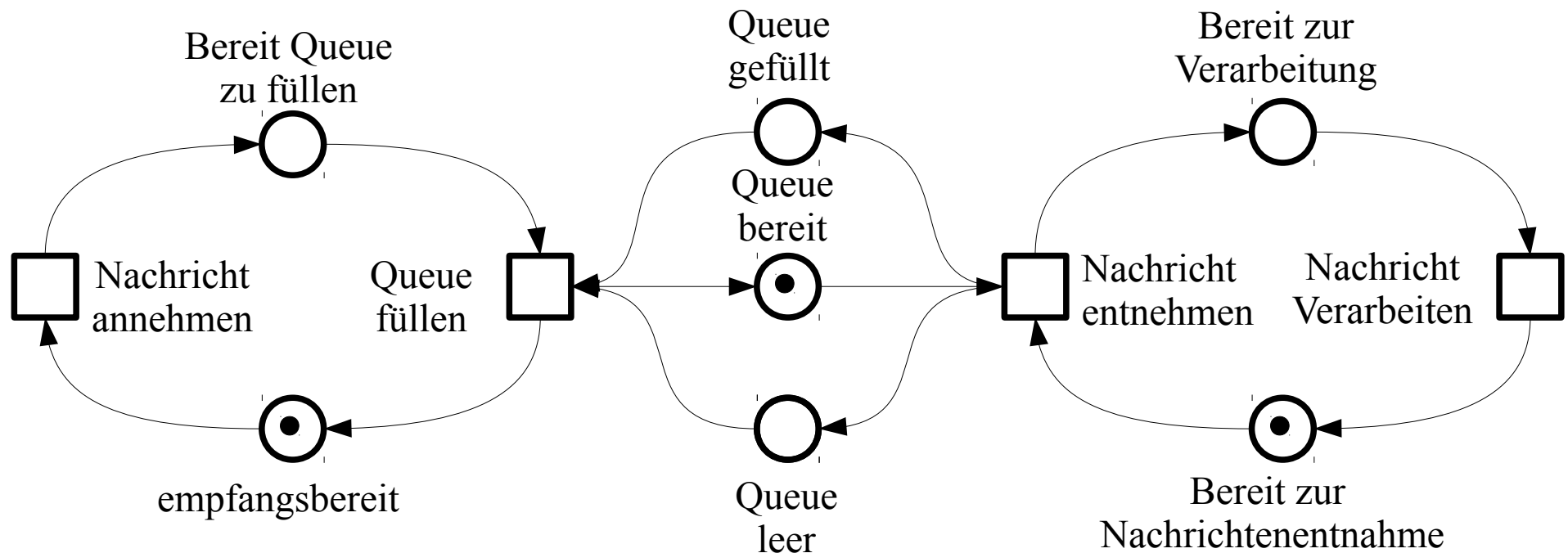
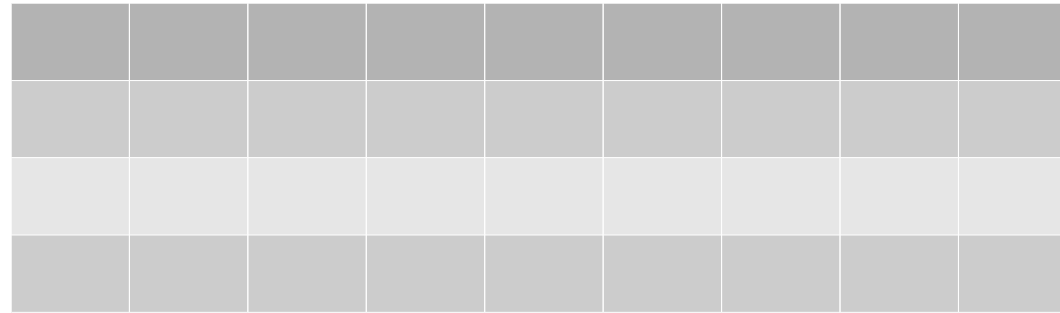
Welche Transition(en) aktiviert ?



Usw. ...

Nichtdeterministische Auswahl von Transition: Beispiel

Softwarekonstruktion
WS 2016/17



1.3 Petrinetze

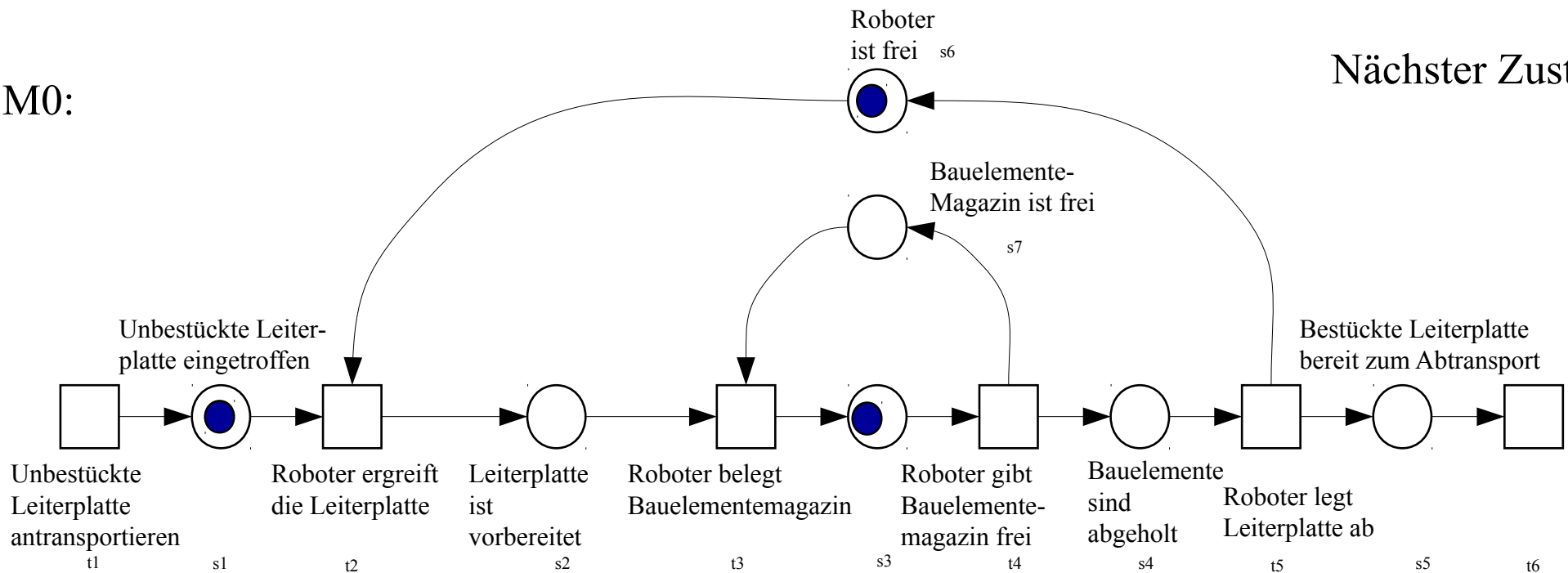
75

Nichtdeterministische Auswahl von Transition: Beispiel

Andere Startmarkierung M0 für nicht-deterministische Verzweigung:

Nr.	s1	s2	s3	s4	s5	s6	s7	Schaltungen
M0	1	0	1	0	0	1	0	
M1								
M2								

M0:



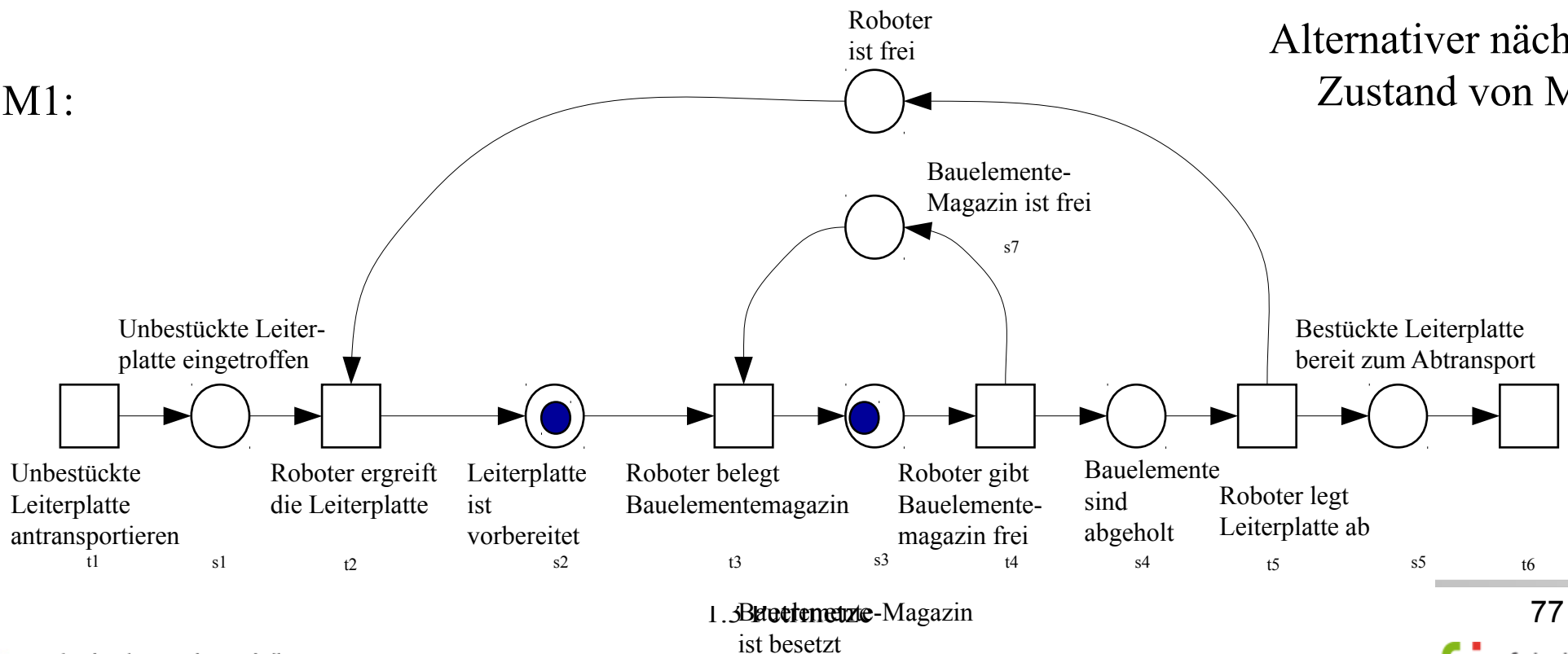
Nächster Zustand

1.3 Petri-Netze
Bauelemente-Magazin ist besetzt

Nichtdeterministische Auswahl von Transition: Beispiel

Nr.	s1	s2	s3	s4	s5	s6	s7	Schaltungen
M0	1	0	1	0	0	1	0	t2->M1
M1	0	1	1	0	0	0	0	
M2								

M1:



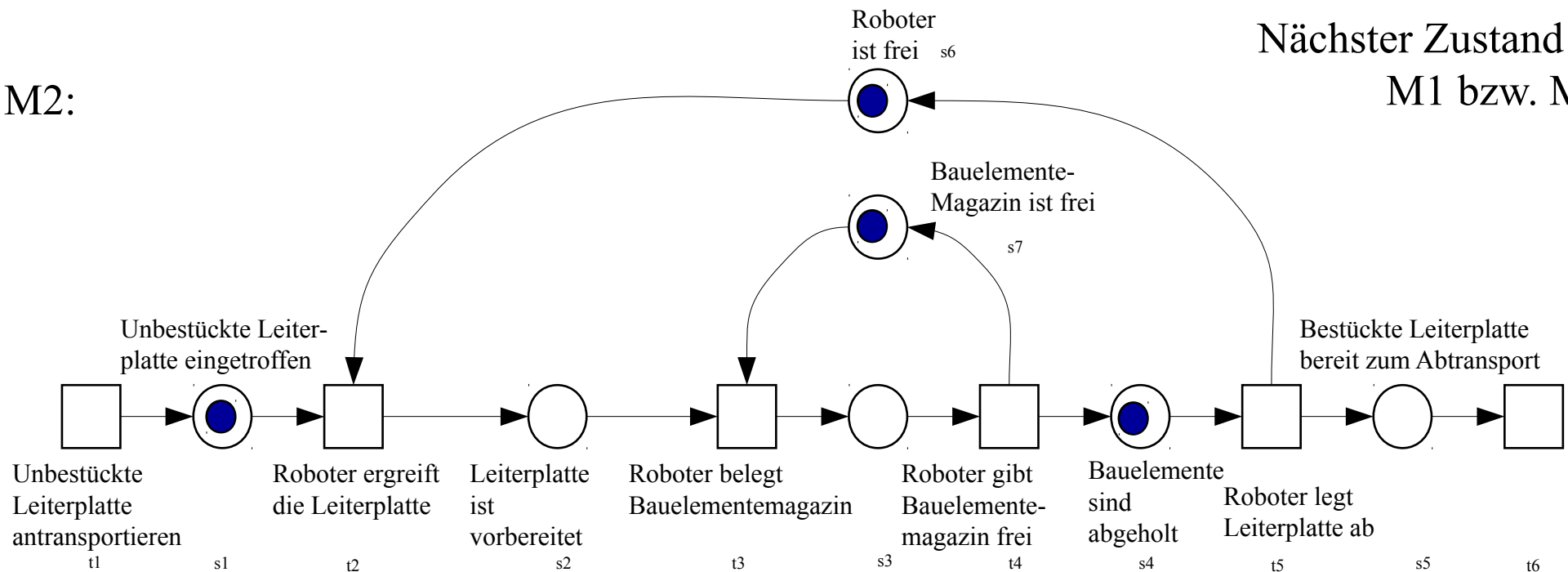
Alternativer nächster Zustand von M0 ?

Nichtdeterministische Auswahl von Transition: Beispiel

Nr.	s1	s2	s3	s4	s5	s6	s7	Schaltungen
M0	1	0	1	0	0	1	0	t2->M1 t4->M2
M1	0	1	1	0	0	0	0	
M2	1	0	0	1	0	1	1	

M2:

Nächster Zustand von
M1 bzw. M2 ?

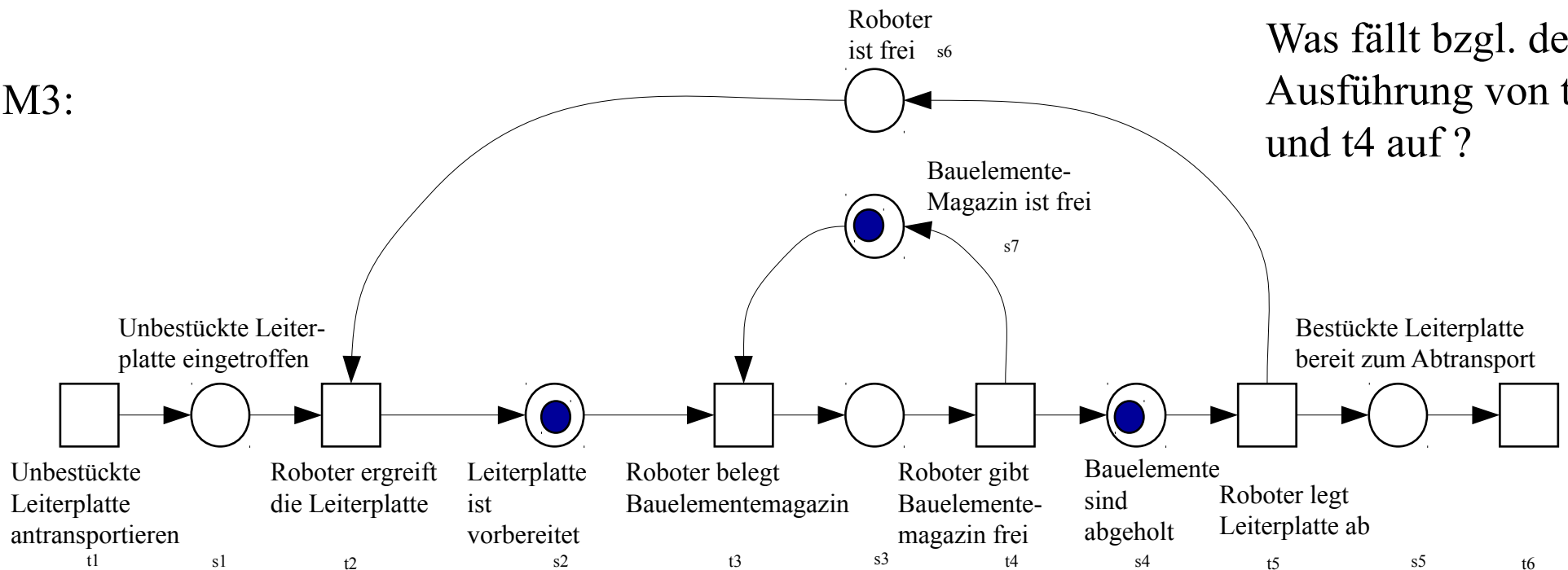


1. Bauelemente-Magazin
ist besetzt

Nichtdeterministische Auswahl von Transition: Beispiel

Nr.	s1	s2	s3	s4	s5	s6	s7	Schaltungen
M0	1	0	1	0	0	1	0	t2->M1 t4->M2
M1	0	1	1	0	0	0	0	t4->M3
M2	1	0	0	1	0	1	1	t2->M3 t5->M4

M3:



Was fällt bzgl. der Ausführung von t2 und t4 auf?

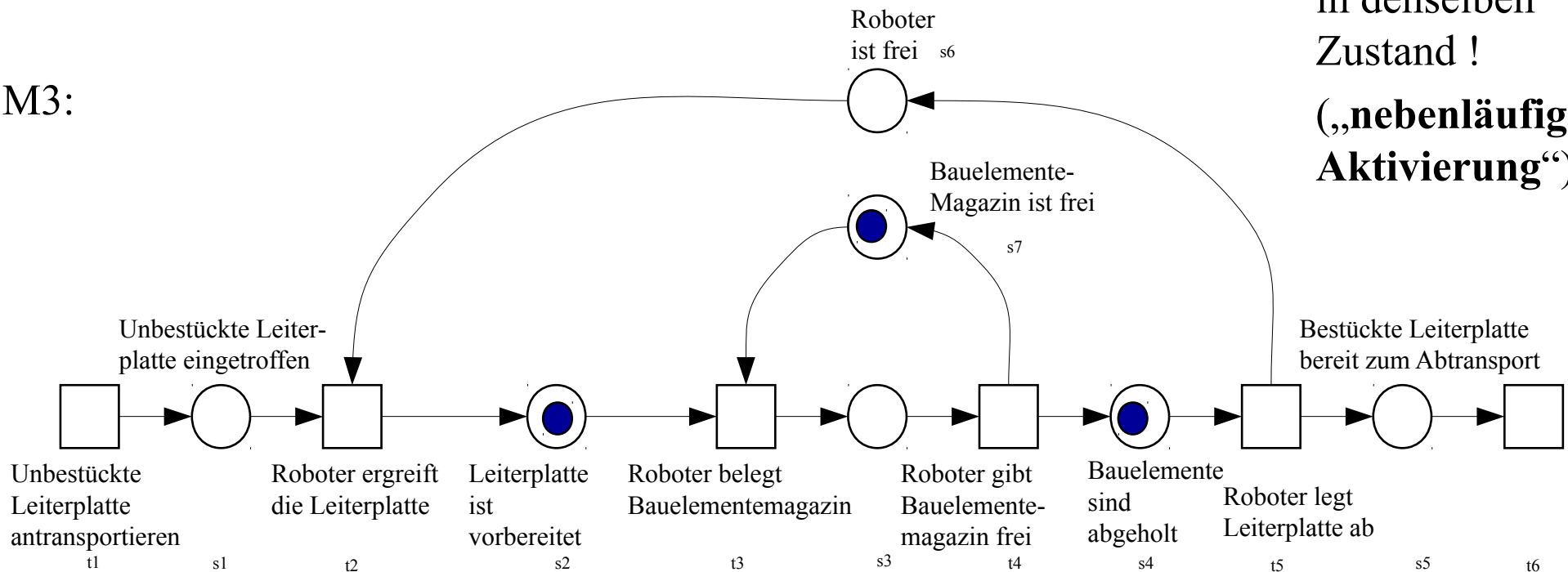
Nichtdeterministische Auswahl von Transition: Beispiel

Nr.	s1	s2	s3	s4	s5	s6	s7	Schaltungen
M0	1	0	1	0	0	1	0	t2->M1 t4->M2
M1	0	1	1	0	0	0	0	t4->M3
M2	1	0	0	1	0	1	1	t2->M3 t5->M4

t2 und t4 können
in beliebiger
Reihenfolge
ausgeführt
werden und
resultieren
in denselben
Zustand !

(„nebenläufige
Aktivierung“)

M3:



1. Bauelemente-Magazin
ist besetzt

80

Erreichbarkeit: Notation und Definition

- $M[t>$: bei Markierung M ist Transition t aktiviert ($[>$ symbolisiert Pfeil)

Erreichbarkeit: Notation und Definition

- $M[t>$: bei Markierung M ist Transition t aktiviert ($[>$ symbolisiert Pfeil)
- $M[t> M'$: M' ist direkte Folgemarkierung zur Markierung M nach Schaltung von Transition t

Erreichbarkeit: Notation und Definition

- $M [t>$: bei Markierung M ist Transition t aktiviert ($[>$ symbolisiert Pfeil)
- $M [t> M'$: M' ist direkte Folgemarkierung zur Markierung M nach Schaltung von Transition t
- $M [w>$: Liste von Transitionen $w=[t_1, t_2, \dots, t_n]$ ist iterativ aktiviert unter Markierung M , d.h.: $M [t_1> M_1 [t_2> M_2 \dots [t_n> M_n$

Erreichbarkeit: Notation und Definition

- $M [t >$: bei Markierung M ist Transition t aktiviert ($[>$ symbolisiert Pfeil)
- $M [t > M'$: M' ist direkte Folgemarkierung zur Markierung M nach Schaltung von Transition t
- $M [w >$: Liste von Transitionen $w=[t_1, t_2, \dots, t_n]$ ist iterativ aktiviert unter Markierung M , d.h.: $M [t_1 > M_1 [t_2 > M_2 \dots [t_n > M_n$
- $M [\{t_1, t_2, \dots, t_n\} >$: Liste von Transitionen $[t_1, t_2, \dots, t_n]$ ist in beliebiger Schaltungsreihenfolge iterativ aktiviert unter Markierung M (= alle Permutationen als Schaltfolgen aktiviert; genannt "**nebenläufig aktiviert**")

Erreichbarkeit: Notation und Definition

- $M [t >$: bei Markierung M ist Transition t aktiviert ($>$ symbolisiert Pfeil)
- $M [t > M'$: M' ist direkte Folgemarkierung zur Markierung M nach Schaltung von Transition t
- $M [w >$: Liste von Transitionen $w=[t_1, t_2, \dots, t_n]$ ist iterativ aktiviert unter Markierung M , d.h.: $M [t_1 > M_1 [t_2 > M_2 \dots [t_n > M_n$
- $M [\{t_1, t_2, \dots, t_n\} >$: Liste von Transitionen $[t_1, t_2, \dots, t_n]$ ist in beliebiger Schaltungsreihenfolge iterativ aktiviert unter Markierung M (= alle Permutationen als Schaltfolgen aktiviert; genannt "**nebenläufig aktiviert**")
- $[M_0 > := \{M \mid \exists w \in T^* \text{ mit } M_0 [w > M\}$ (**Erreichbarkeitsmenge** des Systems; die Markierungen $M \in [M_0 >$ heißen **erreichbar**)

Eingabe: Petrinetz. **Ausgabe:** Erreichbarkeitstabelle (vgl. vorletzte Folie).

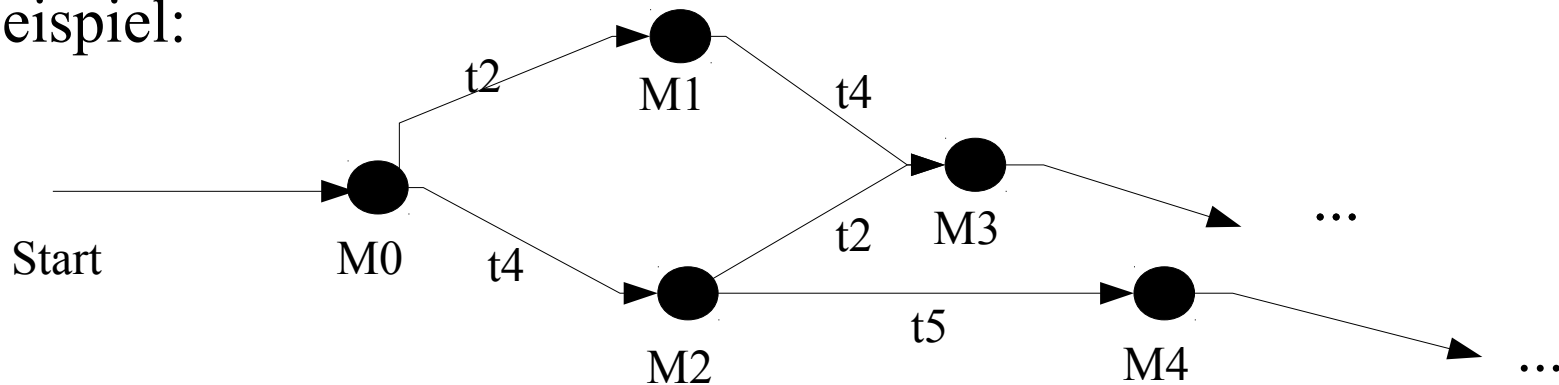
1. Trage in ein Schema mit Spalten „Markierungsnummer“, „Markierung“ und „Schaltungen“ **Anfangsmarkierung M_0** ein.
2. In **aktueller Markierung M_i** für jede **Transition t** : aktiviert ?
 - Falls **t** aktiviert: Berechne Folgemarkierung.
 - Folgemarkierung bereits eine **Markierung M_j** ?
 - Wenn nicht: Benenne Folgemarkierung **M_j** (für ein neues $j > i$) und lege neue Zeile in der Tabelle für **M_j** an.
 - In beiden Fällen: Trage **$M_i[t > M_j$** in Zeile **M_i** , Spalte „Schaltungen“ ein.
3. **M_i** erledigt, falls **alle** Transitionen überprüft.
4. Alle eingetragenen Markierungen erledigt ?
 - **Ja**: Erreichbarkeitsanalyse abgeschlossen
 - **Nein**: Überprüfe die nächste Markierung und fahre bei 2 fort.

Nr.	s1	s2	s3	s4	s5	s6	s7	Schaltungen
M0	1	0	1	0	0	1	0	t2->M1 t4->M2
M1	0	1	1	0	0	0	0	t4->M3
M2	1	0	0	1	0	1	1	t2->M3 t5->M4

Erreichbarkeitstabelle oft als Graph dargestellt:

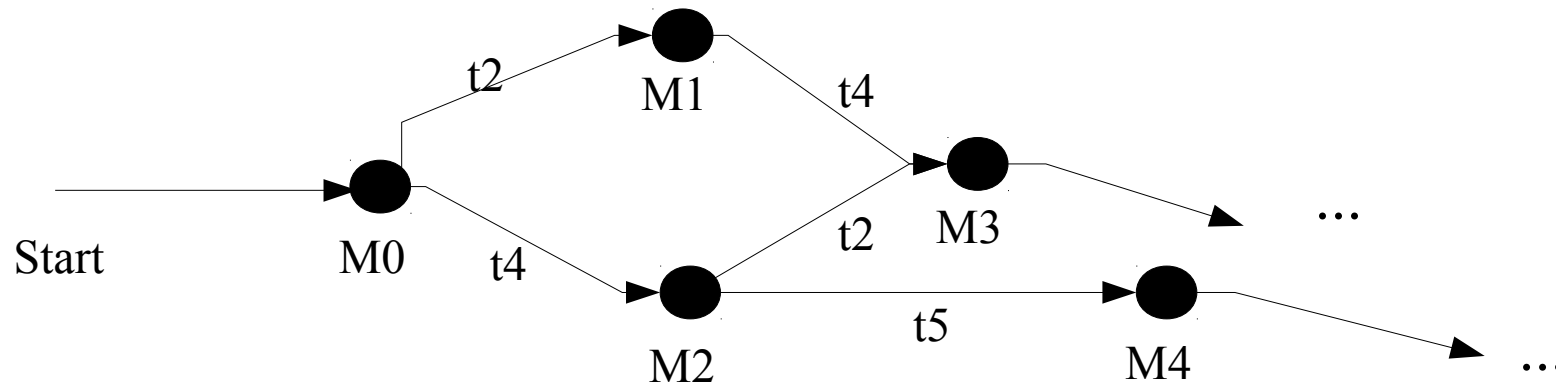
- Knoten: Zustände (linke Spalte; ggf. inkl. Markierungsbelegungen)
- Kanten: Schaltungen (rechte Spalte)

Obiges Beispiel:



Von Erreichbarkeitstabelle zu Event-Log

Nr.	s1	s2	s3	s4	s5	s6	s7	Schaltungen
M0	1	0	1	0	0	1	0	t2->M1 t4->M2
M1	0	1	1	0	0	0	0	t4->M3
M2	1	0	0	1	0	1	1	t2->M3 t5->M4



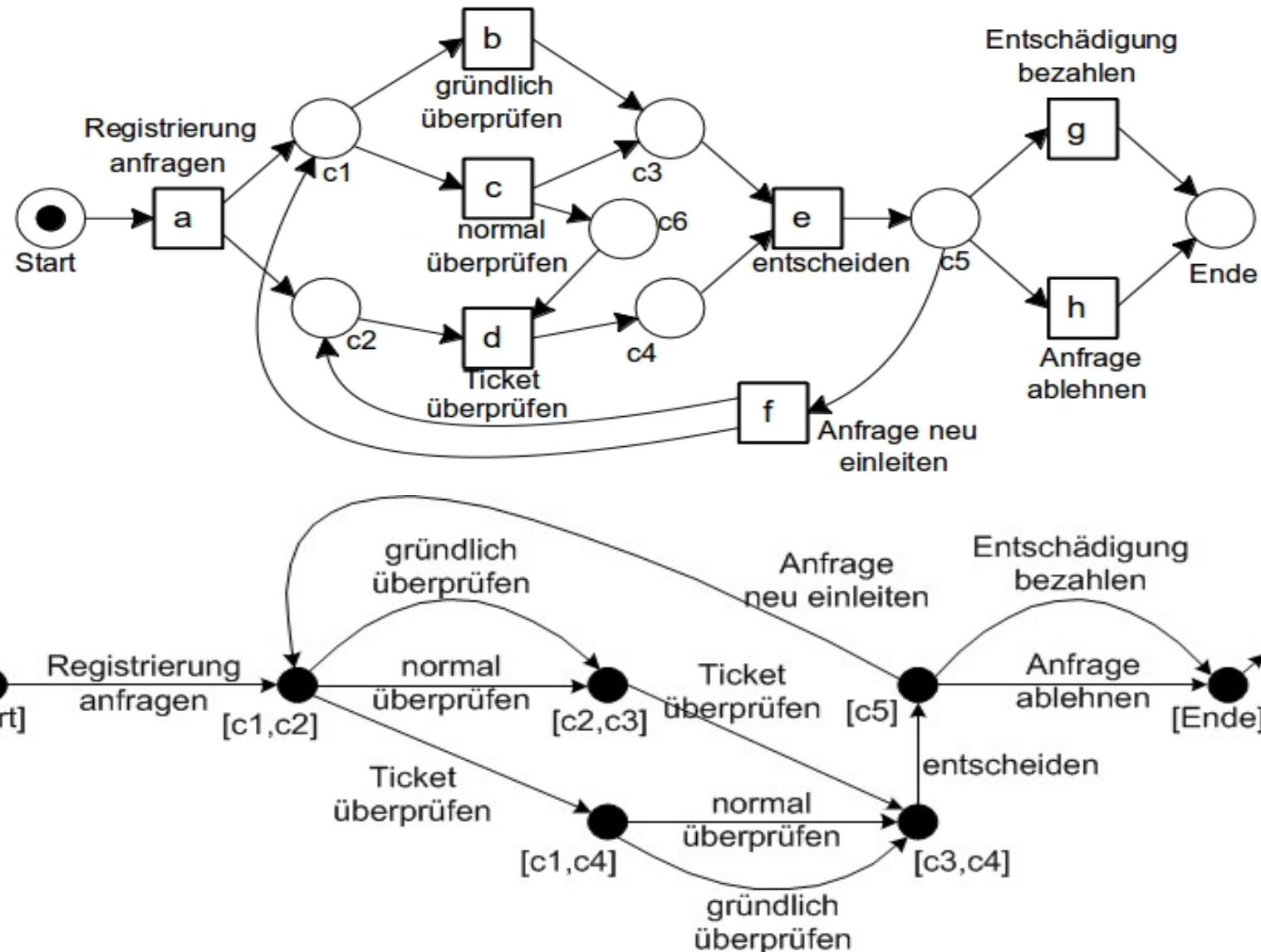
Erzeugtes Event-Log

(= Menge der Folgen der ausgeführten Transitionen):

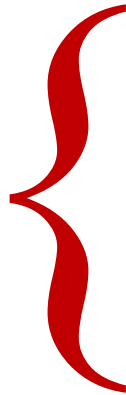
$\{[t2,t4,...],[t4,t2,...],[t4,t5,...], \dots\}$

Erreichbarkeitsgraph: Weiteres Beispiel

NB:
Bezeichnungen der Zustände im Erreichbarkeitsgraph (z.B. [c1,c2]) spiegeln globalen Zustand wieder
→ **mit Markern belegte Stellen im Petrinetz.**



1.4 Petrinetze



Petrinetz Syntax

Ausführung

Analyse von Systemen

Workflow-Netze

Simulation:

- Kann zeigen, dass bestimmte Situationen auftreten können.
- Kann **nicht** zeigen, dass bestimmte Situationen **nicht** auftreten.
- **Ausschnitt** aus Menge aller möglichen Verhalten.
- Keine Aussage über deren **Eintrittswahrscheinlichkeit**.

Verifikation: Beweis von Eigenschaften:

- **Statische Eigenschaften:** Unabhängig von Markierungen, nur von Netztopologie abhängig.
 - z.B. Verklemmungen / Deadlocks
- **Dynamische Eigenschaften:** Abhängig von der Menge erreichbarer Markierungen.
 - Standardhilfsmittel: Erreichbarkeitsgraphen (s.o.)

Sei $P=(S,T,F,K,W,M_0)$ Petrinetz.

Abbildung $B: S \rightarrow \mathbb{N} \cup \{\infty\}$ ordnet jeder Stelle eine „kritische Markenzahl“ zu.

Petrinetz P heißt:

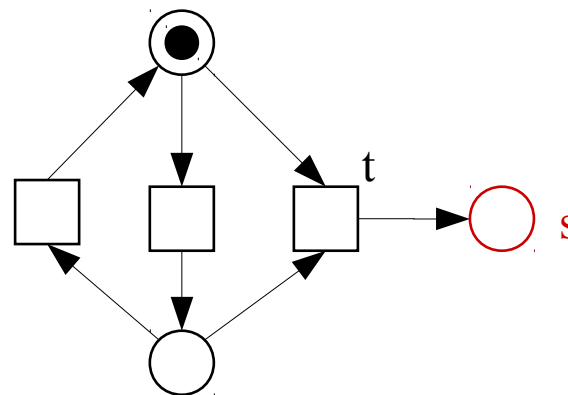
- **B-sicher** (oder **B-beschränkt**), wenn für alle erreichbaren Markierungen Anzahl der Markierungen pro Stelle durch B begrenzt, d.h.:
für alle $M \in [M_0>$ und $s \in S$ gilt: $M(s) \leq B(s)$.
- **1-sicher**, **2-sicher** usw., wenn $B=1$, $B=2$ usw.
- **beschränkt**, wenn es natürliche Zahl b gibt, für die P b -sicher.

Stelle s heißt **b-sicher**, wenn P B -sicher mit $B(s)=b$, und $B(s')=\infty$ für $s' \neq s$.

Unterschied zwischen Kapazität und Sicherheit:

- **Kapazität begrenzt** Stellenmarkierung (a priori-Begrenzung).
- **Sicherheit beobachtet** Stellenmarkierung (a posteriori-Begrenzung).

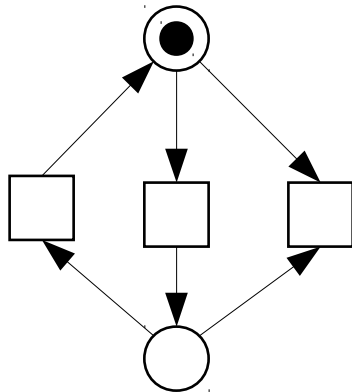
- *Beispiel:* Verkehrsplaner modelliert **Ampelsystem**.
 - An bestimmter Stelle s darf sich nur ein Auto aufhalten.
 - $K(s)=1$: Keine Aussage über Korrektheit der Modellierung.
 - $K(s)=\infty$: In Analyse prüfbar, ob $B(s)=1$.
- *Beispiel:* Transition t soll nie schalten dürfen.
 - Sicherheitseigenschaft: Beobachtungsstelle s und Bedingung $B(s)=0$ hinzufügen.



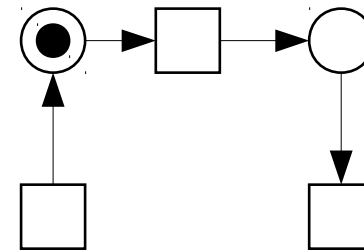
Transition t eines Petrinetz $P=(S,T,F,K,W,M_0)$ heißt:

- **aktivierbar**: In mindestens einer erreichbaren Markierung aktiviert:
existiert $M_1 \in [M_0>$ mit: $M_1[t>$
- **lebendig**: In allen erreichbaren Markierungen **aktivierbar**:
für alle $M_1 \in [M_0>$ gilt: existiert $M_2 \in [M_1>$ mit: $M_2[t>$
- **tot**: In keiner erreichbaren Markierung aktiviert:
für alle $M \in [M_0>$ gilt: $\neg M[t>$

Tot ist nicht logische Negation von lebendig sondern von aktivierbar !



Transition
ist ... ?



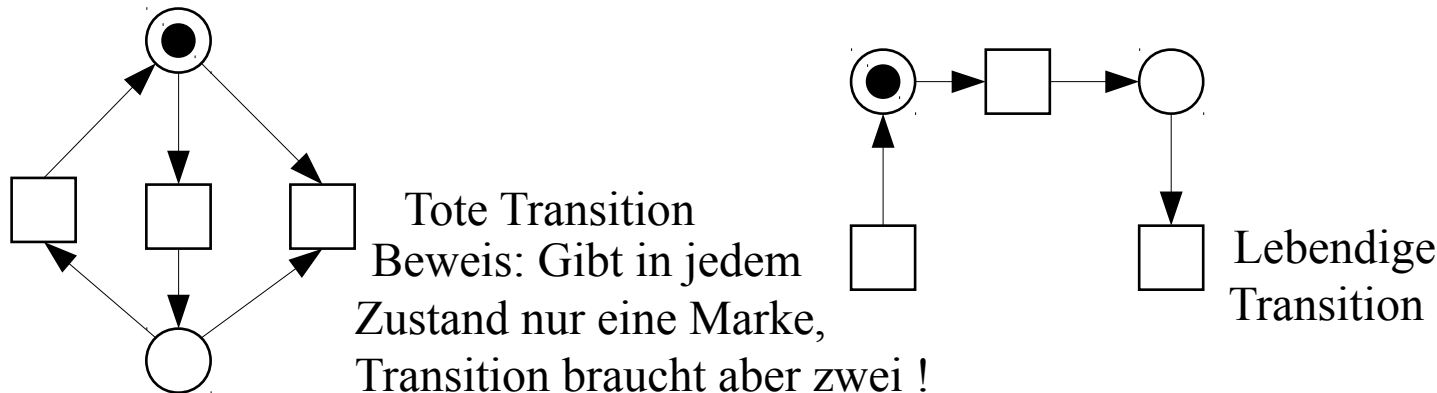
Transition
ist ... ?



Transition t eines Petrinetz $P=(S,T,F,K,W,M_0)$ heißt:

- **aktivierbar**: In mindestens einer erreichbaren Markierung aktiviert:
existiert $M_1 \in [M_0>$ mit: $M_1[t>$
- **lebendig**: In allen erreichbaren Markierungen **aktivierbar**:
für alle $M_1 \in [M_0>$ gilt: existiert $M_2 \in [M_1>$ mit: $M_2[t>$
- **tot**: In keiner erreichbaren Markierung aktiviert:
für alle $M \in [M_0>$ gilt: $\neg M[t>$

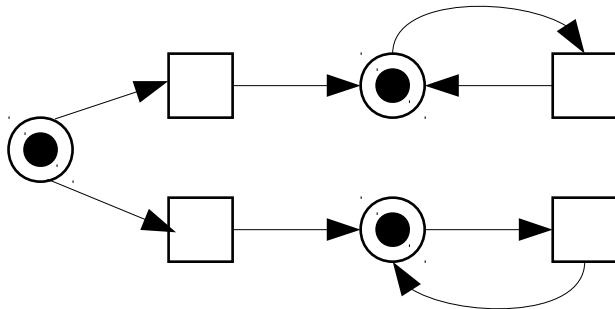
Tot ist nicht logische Negation von lebendig sondern von aktivierbar !



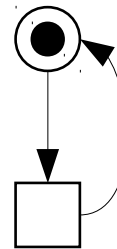
Petrinetz $P=(S,T,F,K,W,M_0)$ heißt:

- **lebendig:** In jeder erreichbaren Markierung ist jede Transition **aktivierbar**:
 $\forall M_1 \in [M_0 > \text{ und } t \in T \text{ gilt: } \exists M_2 \in [M_1 > \text{ mit: } M_2[t >$
- **deadlockfrei:** In jeder erreichbaren Markierung ist mindestens eine Transition aktiviert:
 $\forall M_1 \in [M_0 > \text{ gilt: } \exists t \in T \text{ mit: } M_1[t >$
- **tot:** Keine Transition aktiviert:
 $\forall t \in T: \neg M_0[t >$

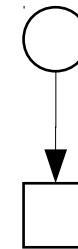
System ist ... ?



System ist ... ?



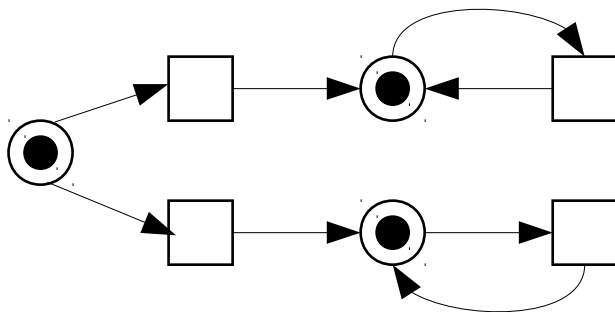
System ist ... ?



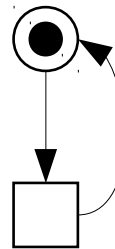
Petrinetz $P=(S,T,F,K,W,M_0)$ heißt:

- **lebendig:** In jeder erreichbaren Markierung ist jede Transition **aktivierbar**:
 $\forall M_1 \in [M_0] \text{ und } t \in T \text{ gilt: } \exists M_2 \in [M_1] \text{ mit: } M_2[t>$
- **deadlockfrei:** In jeder erreichbaren Markierung ist mindestens eine Transition aktiviert:
 $\forall M_1 \in [M_0] \text{ gilt: } \exists t \in T \text{ mit: } M_1[t>$
- **tot:** Keine Transition aktiviert:
 $\forall t \in T: \neg M_0[t>$

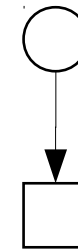
Deadlockfreies System



System ist ... ?



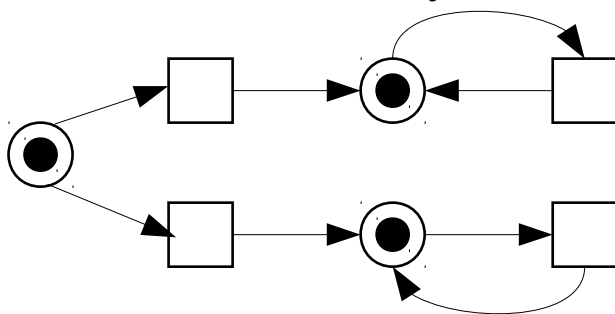
System ist ... ?



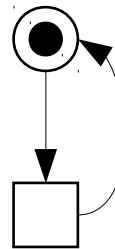
Petrinetz $P=(S,T,F,K,W,M_0)$ heißt:

- **lebendig:** In jeder erreichbaren Markierung ist jede Transition **aktivierbar**:
 $\forall M_1 \in [M_0] \text{ und } t \in T \text{ gilt: } \exists M_2 \in [M_1] \text{ mit: } M_2[t>$
- **deadlockfrei:** In jeder erreichbaren Markierung ist mindestens eine Transition aktiviert:
 $\forall M_1 \in [M_0] \text{ gilt: } \exists t \in T \text{ mit: } M_1[t>$
- **tot:** Keine Transition aktiviert:
 $\forall t \in T: \neg M_0[t>$

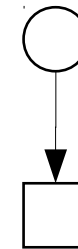
Deadlockfreies System



Lebendiges System



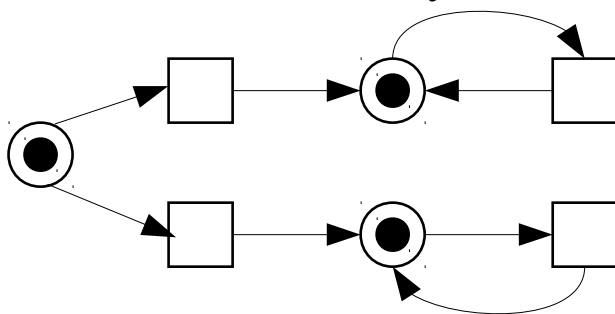
System ist ... ?



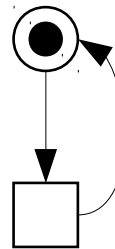
Petrinetz $P=(S,T,F,K,W,M_0)$ heißt:

- **lebendig:** In jeder erreichbaren Markierung ist jede Transition **aktivierbar**:
 $\forall M_1 \in [M_0] \text{ und } t \in T \text{ gilt: } \exists M_2 \in [M_1] \text{ mit: } M_2[t>$
- **deadlockfrei:** In jeder erreichbaren Markierung ist mindestens eine Transition aktiviert:
 $\forall M_1 \in [M_0] \text{ gilt: } \exists t \in T \text{ mit: } M_1[t>$
- **tot:** Keine Transition aktiviert:
 $\forall t \in T: \neg M_0[t>$

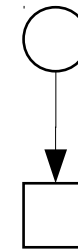
Deadlockfreies System



Lebendiges System



Totes System



Lebendigkeit von Petrinetzen: Anmerkungen

tot: keine Transition aktivierbar, d.h. alle Transitionen tot.

(**Achtung:** Nicht **aktivierte** Transition kann trotzdem **aktivierbar** sein, aber wenn keine Transition im Petrinetz **aktiviert** ist, ist auch keine **aktivierbar** !)



tot: keine Transition aktivierbar, d.h. alle Transitionen tot.

(**Achtung:** Nicht **aktivierte** Transition kann trotzdem **aktivierbar** sein, aber wenn keine Transition im Petrinetz **aktiviert** ist, ist auch keine **aktivierbar** !)



D.h. im Erreichbarkeitsgraph geht von der initialen Markierung keine Kante aus.

- Bedeutet häufig einen Deadlock

tot: keine Transition aktivierbar, d.h. alle Transitionen tot.

(**Achtung:** Nicht **aktivierte** Transition kann trotzdem **aktivierbar** sein, aber wenn keine Transition im Petrinetz **aktiviert** ist, ist auch keine **aktivierbar** !)



D.h. im Erreichbarkeitsgraph geht von der initialen Markierung keine Kante aus.

- Bedeutet häufig einen Deadlock

deadlockfrei: keine erreichbare Markierung tot.

- Berücksichtigung partieller Ausfälle („graceful degradation“).

tot: keine Transition aktivierbar, d.h. alle Transitionen tot.

(**Achtung:** Nicht **aktivierte** Transition kann trotzdem **aktivierbar** sein, aber wenn keine Transition im Petrinetz **aktiviert** ist, ist auch keine **aktivierbar** !)



D.h. im Erreichbarkeitsgraph geht von der initialen Markierung keine Kante aus.

- Bedeutet häufig einen Deadlock

deadlockfrei: keine erreichbare Markierung tot.

- Berücksichtigung partieller Ausfälle („graceful degradation“).

lebendig: alle Transitionen lebendig.

D.h.: in Erreichbarkeitsgraph existiert von jedem Knoten ausgehend für jedes t aus T einen Pfad, in dem t als Label vorkommt.

tot: keine Transition aktivierbar, d.h. alle Transitionen tot.

(**Achtung:** Nicht **aktivierte** Transition kann trotzdem **aktivierbar** sein, aber wenn keine Transition im Petrinetz **aktiviert** ist, ist auch keine **aktivierbar** !)



D.h. im Erreichbarkeitsgraph geht von der initialen Markierung keine Kante aus.

- Bedeutet häufig einen Deadlock

deadlockfrei: keine erreichbare Markierung tot.

- Berücksichtigung partieller Ausfälle („graceful degradation“).

lebendig: alle Transitionen lebendig.

D.h.: in Erreichbarkeitsgraph existiert von jedem Knoten ausgehend für jedes t aus T einen Pfad, in dem t als Label vorkommt.



Lebendig ist nicht logische Negation von tot (sondern stärker).



Gibt viele Varianten dieser Definitionen.

In diesem Abschnitt:

Petrinetze als Grundlage für Geschäftsprozessmodellierung und für Process-Mining.

- Petrinetz-Syntax
- Ausführung
- Analyse von Systemen

Im nächsten Abschnitt:

- Eclipse Modeling Framework (EMF): Standard zur Modellierung.